Computers & Security 173 (2027) 105221

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

A survey of C/C++ third-party component identification: Methods, assumptions, and evaluation reliability☆

Wei Guo, Xixing Li *, Qiang Wei

Information Engineering University, Zhengzhou, 450001, Henan, China

ARTICLE INFO

Keywords:
C/C++
Third-party component identification
Software composition analysis
Binary analysis
Software bill of materials
Software supply-chain security

ABSTRACT

Reusing third-party components reduces duplicate development, but undeclared open-source code may violate license terms, while vulnerable or outdated components can expose downstream products to security risks. When dependency manifests or build records are unavailable, the components, versions, and source fragments present in C/C++ source code, native binaries, and firmware must be recovered from the code itself. We systematically searched five bibliographic databases and selected 75 conference and journal papers from 3712 records. The studies were analyzed along five dimensions: research task, input mode, identification method, failure risk, and experimental evaluation. Most studies address component identification, version identification, or partial-code identification; modified-code identification receives much less attention. Feature matching and multi-feature fusion are the most common methodological approaches. Forty-seven studies primarily use a closed-world setting. Only 13 evaluate unknown components, and only 11 evaluate modified-code reuse. Thus, much of the available evidence comes from controlled experiments in which the correct candidate is present in the reference corpus. Differences in dataset units, output granularity, and evaluation settings further limit direct performance comparisons. Existing methods can identify third-party components under controlled conditions, but evidence remains insufficient for targets outside the reference corpus, realistic code modifications, and changes in build conditions. Future work should emphasize open-set identification, explicit modeling of version and patch relationships, and reproducible, task-specific evaluation protocols.

Contents

1. Introduction 2
2. Literature search, screening, and research method 2
   2.1. Scope and eligibility criteria 3
   2.2. Search and screening process 3
   2.3. Final corpus and data coding 3
3. Research tasks and problem space 4
   3.1. Identification tasks and their relationships 5
   3.2. Input forms and problem boundaries 5
4. Component identification methods 6
   4.1. Common identification process and method taxonomy 6
   4.2. Capabilities and boundaries of the four method families 7
   4.3. Cross-method comparison and evidential limits 8
5. Identification risks, limitations, and failure conditions 8
   5.1. Evidence rules and overall distribution 8
   5.2. Triggers, effects, and available evidence 9
   5.3. Risk interactions, reporting limitations, and failure chains 11
6. Experimental evaluation and evidence reliability 11
   6.1. Datasets, experimental settings, and representativeness 11
   6.2. Metrics, baselines, and cross-study comparability 12

☆ This article is a survey of C/C++ third-party component identification.

* Corresponding author.
E-mail addresses: mageweig@163.com (W. Guo), lxx_scholar@126.com (X. Li), pro_weiqiang@126.com (Q. Wei).

https://doi.org/10.1016/j.cose.2026.105221
Received 26 May 2026; Received in revised form 3 September 2026; Accepted 21 October 2026
Available online 30 October 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

6.3. Public materials, reproducibility, and evidence credibility 12
7. Future research directions 13
7.1. Component, version, and provenance identification in open environments 13
7.2. Provenance under partial-code reuse, modification, and cross-build variation 14
7.3. A task-specific, open, and reproducible evaluation protocol 14
8. Conclusion 14
CRediT authorship contribution statement 14
Declaration of competing interest 14
Appendix A. Supplementary data 15
Data availability 15
References 15

1. Introduction

Reusing open-source components reduces duplicate implementation and shortens development time, but it also introduces license obligations and security risks into downstream products. Programs written in C/C++ remain common in operating systems, desktop software, embedded devices, and Internet-of-Things (IoT) firmware. Undeclared open-source code may violate license terms. Components that contain known vulnerabilities, are no longer maintained, or have been maliciously compromised may also propagate risk to every product that incorporates them. Identifying the third-party components and versions actually present in software is therefore a prerequisite for finding and managing these problems.

Complete dependency manifests and build records can provide component information directly, but many C/C++ systems do not retain such evidence. Developers may copy only part of a codebase, link a component statically, or trim and modify it before reuse. Compilation, optimization, linking, and symbol stripping may then remove or alter component names, version strings, and original code boundaries. For legacy software, closed-source products, and firmware, analysts often have only a source fragment, a binary, or an incomplete software bill of materials (SBOM). They must infer which components are present, which versions were used, and which local code regions originated from those projects.

Existing studies use strings and symbols, hashes and fingerprints, structural or graph matching, learned representations, and combinations of several feature types. These studies do not, however, solve a single uniform problem. Some identify complete components, some distinguish versions, and others trace partially reused or modified code. Their reference corpora, input forms, output granularities, and experimental conditions also vary substantially. A function-level match is not by itself a component-level result. Likewise, high accuracy over a known candidate set does not show that a method can handle a component absent from its reference corpus. Performance comparisons that ignore these task and evaluation differences can overstate practical applicability.

Prior surveys have examined software supply-chain security, managed-language dependencies, code-clone detection, and binary similarity. Many of them rely on package-management metadata or center on pairwise function similarity and vulnerable-code search. They seldom cover, within one framework, source-to-source (S–S), source-to-binary (S–B), and binary-to-binary (B–B) identification for C/C++ components. In contrast, this review treats component identity, version, and source-fragment provenance as the final outputs. It jointly examines research tasks, input conditions, methodological evidence, failure conditions, and the scope supported by experimental results (Reichert and Obelheiro, 2024; Haq and Caballero, 2021; Min and Zhang, 2019; Ponta et al., 2018; Pashchenko et al., 2018; Decan et al., 2019; Kula et al., 2018; Decan et al., 2018a; Kikas et al., 2017; Zimmermann et al., 2019; Decan et al., 2018b; Zerouali et al., 2018; Wittern et al., 2016; Mirhosseini and Parnin, 2017; Xu et al., 2017; Pewny et al., 2015; David and Yahav, 2014; Massarelli et al., 2019; Brumley et al., 2011; S. Kim et al., 2017).

The review addresses four research questions:

- **RQ1:** What objects do existing studies identify, which input modes do they use, and what results do they produce?
- **RQ2:** Which features and matching mechanisms do existing methods use, and under what conditions do these methods operate as intended?
- **RQ3:** Which failure conditions are supported by study-level evidence, and how broadly can the available experimental evidence support claims of applicability?
- **RQ4:** How should methods and evaluation practices evolve to address unknown components, partial-code and modified-code reuse, and cross-build variation?

To answer these questions, we searched IEEE Xplore, Web of Science, Scopus, the ACM Digital Library, and DBLP for studies published from 2010 to 2026. We selected 75 conference and journal papers from 3712 records and analyzed them by research task, input mode, identification method, failure risk, and experimental design. The synthesis shows that existing methods concentrate on component, version, and partial-code identification, but the available evidence does not establish any method family as uniformly superior across tasks and input settings. Most evidence comes from controlled settings in which the correct candidate is present in the reference corpus. Evaluations of out-of-reference targets, realistic code modification, and cross-build variation remain limited, while heterogeneous datasets, output granularities, and evaluation settings restrict direct performance comparison. These gaps motivate open-set decisions, auditable provenance evidence, and task-specific evaluation protocols that distinguish identification tasks and deployment conditions. The complete search records, exclusion decisions, study-level coding, and statistical results are provided in the supplementary material.

This review makes three contributions:

1. **A unified problem and method framework.** We distinguish research tasks, input modes, output granularities, and application scenarios, and organize the common identification process and major method families.
2. **Study-level conclusions from full-text evidence.** Evidence from 75 papers reveals major gaps in handling unknown components and in evaluating modified-code reuse and cross-build behavior.
3. **An auditable evidence base and task-specific evaluation scheme.** We provide the search, screening, coding, and statistical materials and propose evaluation tracks for distinct identification tasks.

The remainder of this paper is organized as follows. Section 2 describes the search, screening, and coding procedures. Section 3 defines the research tasks and problem space. Section 4 analyzes identification methods. Sections 5 and 6 examine failure conditions and the reliability of experimental evidence. Section 7 outlines future directions, and Section 8 concludes the review.

2. Literature search, screening, and research method

We used a systematic and traceable search and screening process for conference and journal papers published between 2010 and August 14, 2026. Reporting was informed by guidelines for systematic

Table 1
Study inclusion and exclusion criteria.

Type	Code	Criterion
Inclusion	I1	The target includes C/C++ source code, a native binary, an executable, or firmware.
Inclusion	I2	The principal task includes component identity, version, open-source reuse, partial or modified source-fragment identification, or recovery of SBOM components from code or binaries.
Inclusion	I3	The paper proposes, improves, or systematically evaluates a concrete component-identification method.
Inclusion	I4	The output reaches component, version, or source-fragment level rather than only deciding whether a pair of functions is similar.
Inclusion	I5	The full text provides enough method and experimental detail to extract features, matching or decision rules, datasets, and evaluation metrics.
Exclusion	E1	The study addresses only another language ecosystem and does not involve C/C++ or native-component identification.
Exclusion	E2	The study addresses only function similarity, binary similarity, or code clones, without a component-level task or evaluation.
Exclusion	E3	The study addresses only vulnerability detection, fuzzing, patch presence, or CVEs, without component or version identification and mapping.
Exclusion	E4	The study discusses only SBOM policy, dependency governance, or supply-chain management, without a technical component-identification method.
Exclusion	E5	The work is a review, tutorial, editorial, book, thesis, poster, abstract, or demonstration, or does not provide a sufficiently complete study.
Exclusion	E6	The work falls outside 2010–2026, is not in English, or is not an eligible conference or journal paper.
Exclusion	E7	The work clearly falls outside computer science, software engineering, or cybersecurity.
Exclusion	E8	The study only applies or integrates existing tools and neither proposes, improves, nor systematically evaluates an identification method.
Deduplication	D1	The record is a duplicate or an earlier version of the same study; the version with the most complete method, information, and experiments is retained.
Exclusion	Other	The study clearly violates an inclusion criterion but does not fit E1–E8; a specific reason must be recorded.

literature reviews and systematic mapping studies, together with the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 statement and its extension for literature searches, PRISMA-S (Page et al., 2021; Rethlefsen et al., 2021; Kitchenham et al., 2009; Petersen et al., 2008; Brereton et al., 2007). This section summarizes the scope, eligibility criteria, screening process, and coding procedure. Platform-specific queries, detailed deduplication rules, study-level exclusion reasons, and the full coding dataset are available in the supplementary material.

2.1. Scope and eligibility criteria

This review covers studies that identify third-party components from C/C++ source code, native binaries, executables, or firmware. Eligible outputs must reach the level of component identity, version, or source fragment. Studies of vulnerability mapping, license analysis, SBOM recovery, or supply-chain analysis are included only when they contain a corresponding identification mechanism.

Table 1 summarizes the criteria used for the final eligibility decision. A study had to satisfy all five inclusion criteria. Records with insufficient information proceeded to further review; no record was excluded merely because an abstract or a single keyword was missing. For each excluded full text, we recorded one primary reason and the supporting evidence.

A boundary study was included only if its function- or code-level matches were aggregated or evaluated at component, version, or source-fragment level and it made a substantive technical contribution to identification. The complete operational rules are reported in Supplementary Sections S1 and S2.

2.2. Search and screening process

The main searches were conducted on August 13–14, 2026, in IEEE Xplore, the Web of Science Core Collection, Scopus, the ACM Digital Library, and DBLP. DBLP supplemented the database searches for major security and software-engineering venues. Search terms combined a target–object block with three task blocks: component identity and software composition analysis, version and variant identification, and partial-code or modified-code reuse. The three query groups were run separately, with overlap permitted, to reduce omissions caused by long queries and platform-specific syntax. We did not retain an auditable forward- or backward-citation-search log and therefore do not report citation chasing as an independent retrieval source. Verbatim queries, search fields, restrictions, and hit counts for each subquery are provided in Supplementary Section S1.

The five databases returned 3712 records. Within-source DOI deduplication removed 553 records, leaving 3159. Cross-source DOI deduplication removed another 600, leaving 2559. Deduplication by normalized title removed 167 more records and produced 2392 unique records. Fig. 1 summarizes the complete retrieval and screening process and reports the number of records retained at each stage.

Title and abstract screening used two independent channels: a Python rule set and semantic assessment by DeepSeek-V4-Flash. The Python channel assigned `Include`, `Exclude`, or `Unclear` from publication year, document type, abstract completeness, and predefined combinations of title, abstract, and keyword terms. The model applied the same scope and exclusion criteria while reading each title and abstract. A record entered the candidate set directly only when both channels returned `Include`; it was excluded at this stage only when both returned `Exclude` with an explicit reason. All other records received conservative review. A Python-only decision was never sufficient for exclusion. During full-text screening, model output was used only to organize eligibility evidence and never determined eligibility on its own. The screening script, model prompt, run configuration, and record-level results for all 2392 records are included in the supplementary material.

We obtained and uniquely matched the full texts of 103 candidate papers. A model first assisted with extracting evidence relevant to eligibility. One researcher then checked all 103 full texts against the same criteria. Studies at the boundary between component identification, general code similarity, and downstream security applications, as well as studies with conflicting evidence in the first assessment, received a second review by the same researcher. Full-text screening excluded 28 papers and retained 75. The primary exclusion reason and supporting full-text evidence for each excluded paper are reported in Supplementary Section S2. Because two independent human screening records were not available, we do not report inter-rater agreement and treat the subjectivity of a single reviewer as a threat to validity.

2.3. Final corpus and data coding

The final corpus contains 75 studies: 52 conference papers and 23 journal articles. Although the search period begins in 2010, no study published in 2010–2012 passed the final eligibility assessment. As shown in Fig. 2, the included papers were published between 2013 and 2026; publication volume increased noticeably after 2023 and

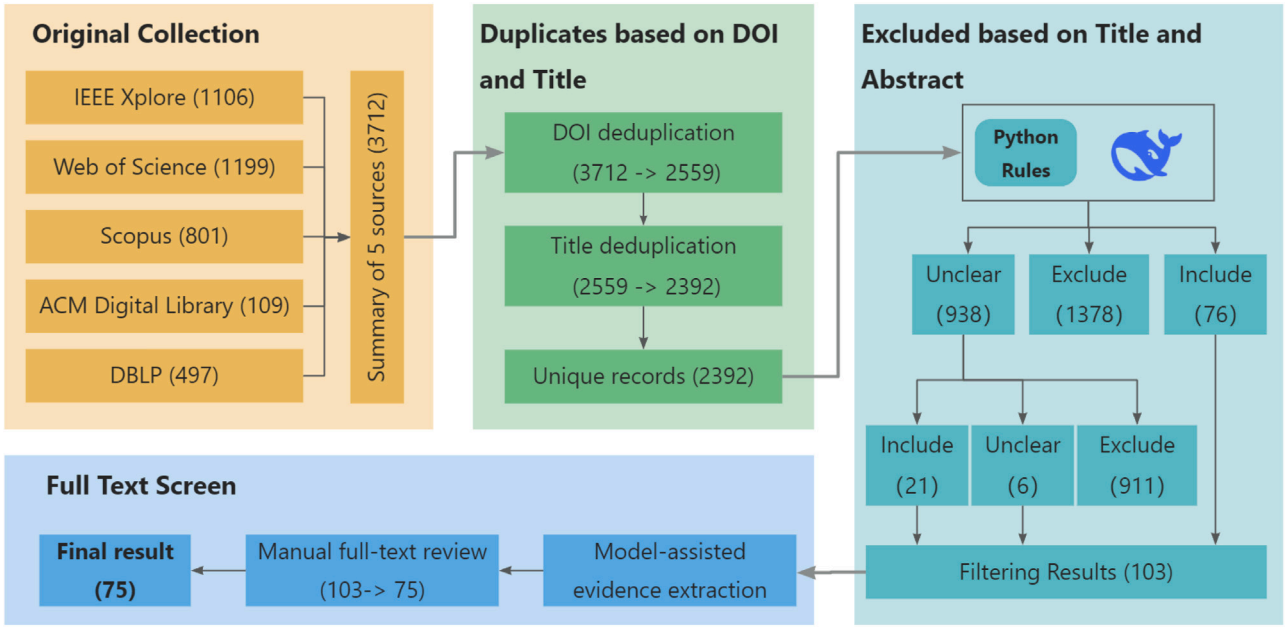


Fig. 1. Integrated literature search and screening process.

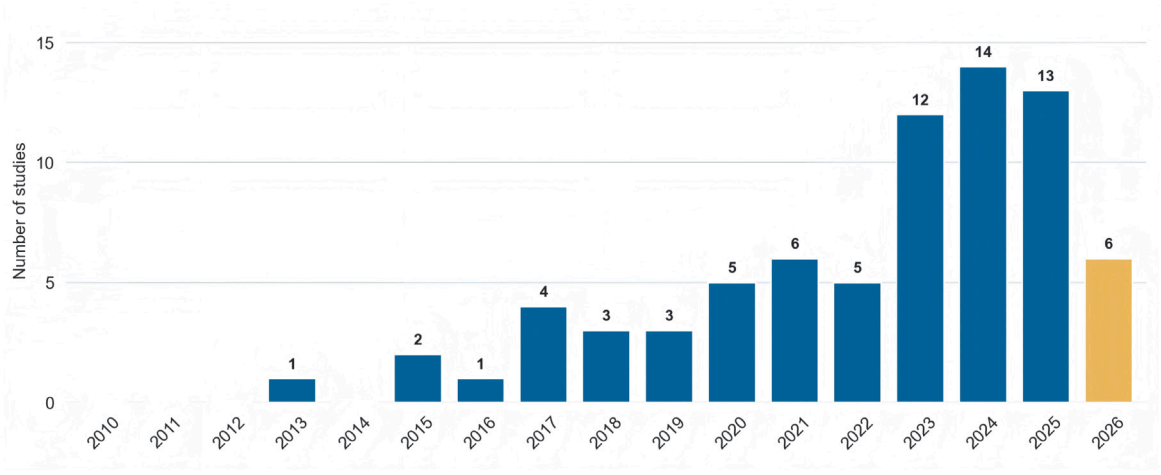


Fig. 2. Publication-year distribution of the included studies. The 2026 count covers publications indexed by August 14.

peaked in 2024. The six papers counted for 2026 reflect only the search coverage through August 14 and should not be compared directly with complete calendar years.

Full-text data extraction used a study-level coding scheme with five groups of fields: research tasks and input modes; methods and features; experimental data and evaluation settings; risks and limitations; and public research materials and reproducibility. Tasks, input modes, evaluation environments, and application scenarios were coded separately. Dataset sizes were retained separately for projects, components, versions, functions, binaries, and firmware. Metrics and the `Closed-World`, `OpenWorld`, and `Mixed` settings were also recorded as distinct fields. Each study retained a stable P-number; its bibliographic mapping is provided in Supplementary Section S4.

One researcher checked the coded fields and their page- and section-level evidence using a uniform template and resolved conflicts or insufficient evidence. Information not reported by the paper was coded as `NotReported`; evidence that remained insufficient was coded as `Unclear`. Neither code was converted into a negative finding. Risk evidence was further separated into direct experiments, author-reported limitations, and review-level analytical judgments. After manual verification, the dataset of 75 studies was frozen and used to generate all statistics in this article. The reference list contains the complete set of included papers.

This review remains subject to incomplete query coverage, differences in database indexing, the absence of auditable citation chasing, model and PDF-parsing errors, and single-researcher full-text decisions. We mitigated these risks through multiple databases and task-specific queries, conservative review, and full-text evidence with page references. Search logs, prompts, human decisions, complete coding, and statistical outputs are retained in the supplementary material.

3. Research tasks and problem space

This section defines the problem space along three dimensions: research task, input form, and evaluation environment. We define a third-party component as a software unit supplied by an entity other than the developer of the target software and reused either in whole or in part. Third-party libraries (TPLs) are the most common form. Component identification, version identification, partial-code identification,

Table 2
Definitions and distribution of research tasks (multi-label coding).

Task	Operational criterion	Studies	Share
Component identification	Determine whether the target contains a particular third-party component and assign its component identity. Local source attribution alone does not establish component-level presence.	74	98.7%
Version identification	Return an exact version, a candidate-version set, or a version ranking. Direct extraction of a version string still counts as version information, but should be evaluated separately from inference based on code differences.	57	76.0%
Partial-code identification	Attribute a file, module, function group, or code fragment to a component or source project. General function similarity does not automatically satisfy this definition.	50	66.7%
Modified-code identification	Recover the original component, version, or source fragment after the target developer has modified reused public component code. Compilation and architecture changes do not automatically count.	16	21.3%

and modified-code identification are distinct but overlapping tasks. S–S, S–B, and B–B describe the forms of the reference and target data. `ClosedWorld`, `OpenWorld`, and `Mixed` describe the relationship between test targets and the reference corpus. Vulnerability mapping, license analysis, SBOM recovery, and supply-chain analysis are treated as application scenarios. This framework answers RQ1.

3.1. Identification tasks and their relationships

Table 2 provides operational definitions for the four tasks. The labels are not mutually exclusive, so their counts sum to more than 75.

The four tasks are related but not equivalent. Component identification asks whether a target contains a particular component and returns its component identity. Partial-code identification attributes a file, function group, or fragment to its source; attribution of a local fragment does not establish component-level presence. Version identification generally presupposes component identity and relies on version-specific functions, additions and deletions, patch regions, version strings, or evolutionary relationships to distinguish closely related releases. In 42 of the 57 version-identification studies, the correct version was guaranteed to be present in the reference corpus. Their results mainly measure discrimination among known candidates and do not establish behavior when the actual version is absent.

Partial-code identification requires local matches to be aggregated into a component- or source-fragment conclusion. Generic function similarity is insufficient. V1SCAN and MOVERY (Woo et al., 2023, 2022), for example, use vulnerability analysis as an application but also identify the provenance of code fragments and therefore fall within the review scope. Modified-code identification requires an actual modification made by the target developer to reused public source code. Compiler, optimization, architecture, stripping, and ordinary malware variation do not automatically qualify. Eight of the 16 studies that claim modified-code identification evaluated modified-code reuse. Three additional studies evaluated such changes without defining them as a primary task, giving 11 studies with relevant experiments across the full corpus. Task claims and experimental coverage must therefore be reported separately.

Fig. 3 shows that component identification often underpins version and local-provenance analysis. Sixty-nine studies cover at least two tasks. The largest combination, component identification plus version identification plus partial-code identification, contains 25 studies. One study traces only local code provenance and does not make a component-level decision. The 13 studies labeled with all four tasks do not necessarily provide adequate evidence for modified-code reuse; the types, magnitude, ground truth, and actual experimental coverage of modification must still be inspected.

We also tested whether two boundary studies altered the overall results. After removing V1SCAN and MOVERY, the maximum absolute change in any major statistic was 2.4 percentage points. The rank order of method classes, input modes, and evaluation environments was unchanged. The proportion of studies that experimentally evaluate modified-code reuse changed from 11/75 (14.7%) to 9/73 (12.3%). The two papers therefore do not determine the main conclusions. Full sensitivity results appear in Supplementary Section S7.

Table 3
Cross-distribution of research tasks and evaluation environments.

Task	Closed	Open	Mixed	Total
Component identification	47 (63.5%)	22 (29.7%)	5 (6.8%)	74
Version identification	37 (64.9%)	16 (28.1%)	4 (7.0%)	57
Partial-code identification	28 (56.0%)	17 (34.0%)	5 (10.0%)	50
Modified-code identification	7 (43.8%)	7 (43.8%)	2 (12.5%)	16

3.2. Input forms and problem boundaries

We use three input modes, which may overlap. S–S compares reference source code with target source code. S–B derives features from reference source code and uses them to identify a binary target. B–B compares reference and target binaries. Among the 75 studies, 20 use S–S, 37 use S–B, and 22 use B–B. Seventy-one use one mode and four use two. Fig. 4 cross-tabulates these modes with the four research tasks.

The modes expose different information. S–S can use names, syntax, file structure, and commit history, but requires access to target source code. Nine of the 16 modified-code studies use S–S, so much of the evidence for this task depends on source-level information. S–B is the most frequent mode and includes 30 version-identification studies, but it must cope with source information that disappears or changes during compilation, optimization, and linking. B–B avoids an explicit source-to-binary correspondence, yet depends more heavily on similarity between the reference and target build conditions, including compiler, architecture, optimization, and linking configuration. Available evidence does not establish one mode as uniformly superior, and performance cannot be compared across modes without controlling the task and experimental conditions.

The `ClosedWorld`, `OpenWorld`, and `Mixed` labels further characterize the relationship between targets and the reference corpus. A study is `ClosedWorld` when targets come from a known component set or the experiment guarantees that the correct candidate is in the reference corpus. A study is `OpenWorld` when targets are obtained independently from an application store, firmware platform, or code repository not used to construct the reference corpus and may include out-of-reference components. Studies containing both settings are `Mixed`. Of the 75 studies, 47 are `ClosedWorld`, 23 are `OpenWorld`, and five are `Mixed`. Table 3 breaks down these settings by task.

Evaluation remains dominated by known candidates. Of the 57 version studies, 37 use `ClosedWorld`, and 42 guarantee that the correct version is present. Across all 75 studies, 21 systems support a final component-level `Unknown`, no-match, or rejection output; 37 do not, and 17 do not report this capability. An `OpenWorld` data source is not equivalent to unknown-component handling. Real firmware or applications establish where targets came from. Only an ability to reject every reference candidate establishes that the system can handle an

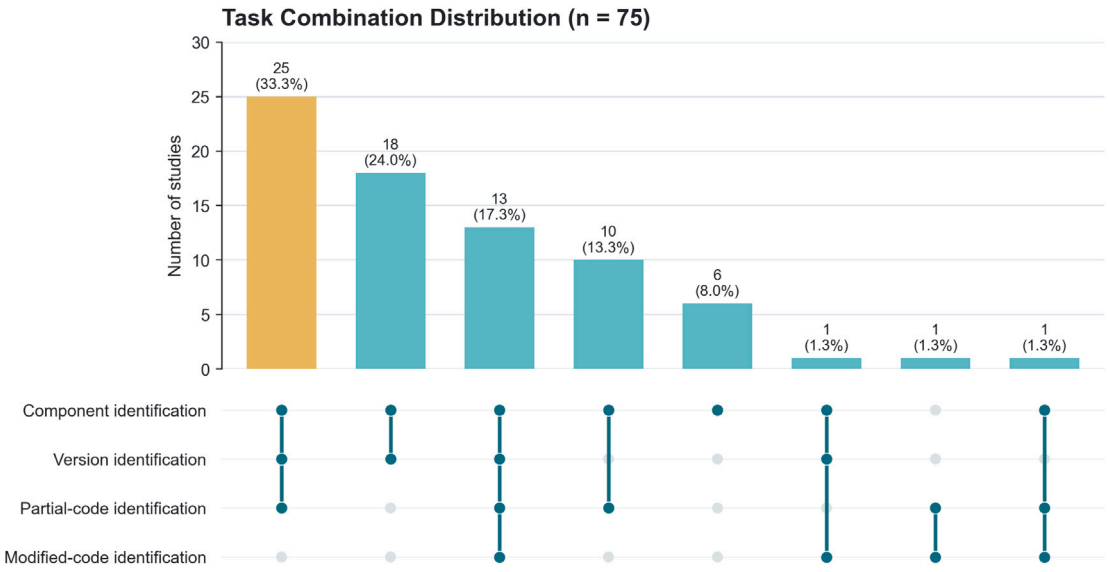


Fig. 3. Distribution of research-task combinations.

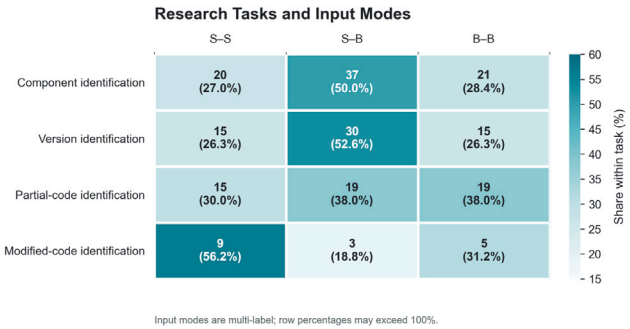


Fig. 4. Cross-distribution of research tasks and input modes. Input modes are multi-label.

unknown component. Sections 5 and 6 examine the resulting risks and evaluation requirements.

SBOM recovery seeks a list of components and versions. Vulnerability mapping requires correct component, version, or patch status. License analysis also needs the copied extent and source of code, while firmware analysis commonly faces multiple architectures, static linking, and vendor customization. These applications do not alter the task definitions, but they change the consequences of each identification error.

Answer to RQ1. Existing work is built around component identification and extends to version and local-provenance analysis. Partial-code and modified-code identification capture more difficult forms of reuse and may overlap with component and version identification. S–B is the most common input mode, but S–S, S–B, and B–B provide different information and cannot be compared without accounting for their conditions. The literature remains oriented toward `ClosedWorld` and known candidates, with limited experimental evidence for unknown-component rejection and modified-code reuse.

4. Component identification methods

C/C++ component identification generally consists of reference-corpus construction, feature extraction, candidate matching, and evidence aggregation, followed by an output at component, version, or source-fragment level. We classify the 73 studies with a clear primary method into four mutually exclusive groups: feature matching, structural and graph matching, learned representations, and multi-feature fusion. Two systematic-evaluation or multi-method comparison studies remain in the corpus but are not assigned a method class. The analysis focuses on the evidence used by each family, its decision process, and its scope of applicability. It does not rank families by publication count or isolated performance values. This section answers RQ2.

4.1. Common identification process and method taxonomy

Fig. 5 summarizes the common TPL-identification process. On the reference side, features from known components and versions are indexed. Comparable code representations are extracted from the target. Candidate generation, matching, and ranking then combine the two sides. Methods differ primarily in their feature representation, candidate-generation strategy, matching mechanism, and ranking rules.

Intermediate matches may denote components, functions, files, or local regions and should not be conflated with the final component-level conclusion. A method must filter common code and aggregate local evidence into component identity, version, or source provenance. When evidence is insufficient, it should be able to return `Unknown`, no match, or ambiguity rather than force the highest-scoring candidate. Reference coverage, feature observability after modification or compilation, evidence aggregation, and rejection rules are therefore common dependencies across method families.

Table 4 assigns each of the 73 method studies according to its dominant final-decision mechanism. A study that uses function names, control-flow graphs, and constants is classified as multi-feature fusion when these evidence types jointly determine its conclusion. It is classified as structural and graph matching when graph relations themselves are the principal matching object, and as a learned-representation method when a learned embedding or classifier determines similarity or labels. This primary label is mutually exclusive; it does not imply that a study uses only one feature type.

Across all 75 papers, textual or lexical features appear in 40 studies, structural or graph features in 30, instructions or constants in 23, metadata or history in 18, semantic or learned features in 15, and hashes or fingerprints in 14. These are multi-label counts and indicate only which information a paper uses. For example, 15 studies use semantic or learned features, but only four rely on learned representations as the primary decision mechanism. Similarly, 30 use structural or graph features, while only eight are classified as structural and graph matching.

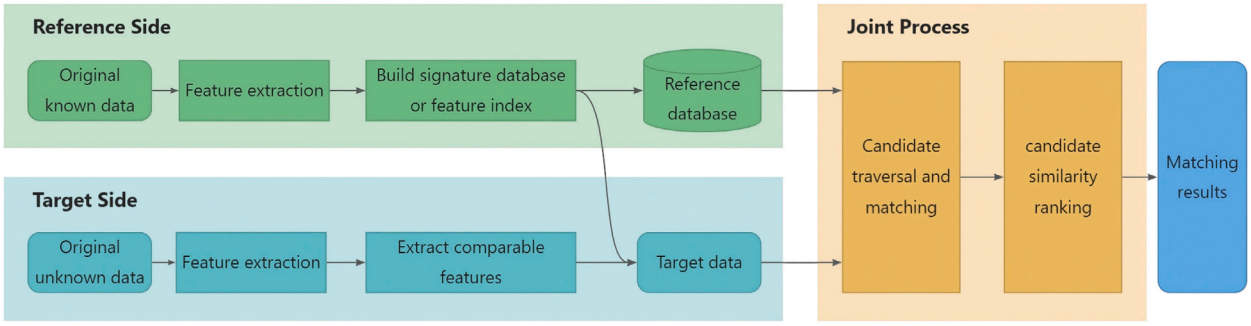


Fig. 5. General workflow for C/C++ third-party component identification.

Table 4
Primary identification methods and their distribution ($n = 73$).

Method family	Core mechanism and typical evidence	Studies (share)
Feature matching	Exact or approximate matching of strings, symbols, version markers, hashes, constants, opcodes, or historical information.	31 (42.5%)
Structural and graph matching	Graph matching, distance computation, or structural aggregation over control flow, calls, dependencies, or regions.	8 (11.0%)
Learned representations	Vector distance, class probability, or learned ranking derived from instructions, disassembly, or function embeddings.	4 (5.5%)
Multi-feature fusion	Staged filtering, weighting, or multi-stage decisions that combine textual, structural, hash, embedding, or metadata evidence.	30 (41.1%)
Total	–	**73 (100.0%)**

Percentages use the 73 studies with a clear primary method as the denominator and sum to 100.1% because of rounding. P2 (Liu et al., 2026) is a systematic evaluation, and P11 (Wang et al., 2025) compares and enhances several methods. Both remain in the 75-paper corpus but are not treated as additional method classes.

Table 5
Core operating conditions of the primary method families.

Method family	Principal operating conditions	Typical risk when a condition fails
Feature matching	The reference corpus covers the true source; features remain observable and discriminative; common code is filtered.	Out-of-reference false negatives or false attribution, false positives from common features, and feature loss after modification or compilation.
Structural and graph matching	Structure can be recovered and remains aligned; local matches can be aggregated to components.	Topology changes caused by inlining, optimization, or trimming, and source ambiguity from common subgraphs.
Learned representations	Training, reference, and test data are separated; representations remain stable; similarity is calibrated.	Data leakage, distribution shift, and misuse of similarity as confidence.
Multi-feature fusion	Candidate generation preserves recall; evidence sources are complementary; weights, thresholds, and aggregation transfer across datasets.	Early-stage omission, double counting, and propagation of errors across stages.
Common conditions	Ground truth, output granularity, and reference scope are explicit; `Unknown` or tied candidates are permitted.	Function matches promoted to component conclusions, or forced labels for versions and out-of-reference targets.

The conditions discussed below are supported by three kinds of evidence: explicit author statements, conditions embodied in experimental settings, and our analysis of the common identification process. The first two characterize what a paper reports or evaluates. The third explains where a method may depend on a particular condition; it is not evidence that the original paper experimentally validated that dependency. Table 5 provides a cross-family framework. Section 5 then uses direct experiments and author-reported limitations to determine whether violations of these conditions have observed consequences.

4.2. Capabilities and boundaries of the four method families

Feature matching. OSSFP (Wu et al., 2023) combines function fingerprints with metadata, CENTRIS (Woo et al., 2021) uses Trend Micro Locality Sensitive Hash (TLSH) and weighted rules, and MOVERY (Woo et al., 2022) matches vulnerability- and patch-related code lines. The family spans source-level dependency and reuse identification (Zhang et al., 2025; S. Xu et al., 2025; He et al., 2025; Hai et al., 2025; R. Li et al., 2024; Reid and Mockus, 2023), binary component or function attribution (Tang et al., 2020; Zhang et al., 2024a; Tang et al., 2018; Dominguez-Alvarez et al., 2023; Akabane and Okamoto, 2020; D. Kim et al., 2017; Nouh et al., 2017; Kim and Choi, 2025; Stojanovic et al., 2015), and version identification, firmware or mobile analysis, and SBOM recovery (Zhang et al., 2024b, 2026; W. Zhang et al., 2018; Zhang and Yu, 2023; Yu et al., 2023; Rhee et al., 2024; W.-D. Zhang et al., 2018; Hu et al., 2020; Akabane and Okamoto, 2021). Such evidence is easy to locate and index, but it must remain visible and discriminative in the target. Symbol stripping, common code, source modification, and build variation can weaken that requirement. Of the 31 studies, 30 address component identification, 25 version identification, 20 partial-code identification, and eight modified-code identification.

Structural and graph matching. P449 (Qiu et al., 2015) uses graph isomorphism to identify library functions, P1079 (Qiu et al., 2016) uses reduced execution-flow graphs, and CentSCA (Y.-T. Hu et al., 2026) uses function-call graphs and centrality. Other studies use graph-based homology models, structural similarity in native libraries, or function relations for supply-chain analysis (Jia et al., 2024; Kalysch et al., 2018; Sun et al., 2020). Structural relations provide context when symbols are absent, but inlining, optimization, partial import, and shared subgraphs can alter or confound them. Full-graph matching can also be expensive. Seven of the eight studies use `ClosedWorld`; two support unknown rejection, four do not, and two do not report it. Evidence in open environments is therefore limited.

Learned representations. P505 (Haile and Havens, 2020) performs supervised classification over disassembly features, BinCoFer (Zou et al., 2025) combines function embeddings with weighted aggregation, and

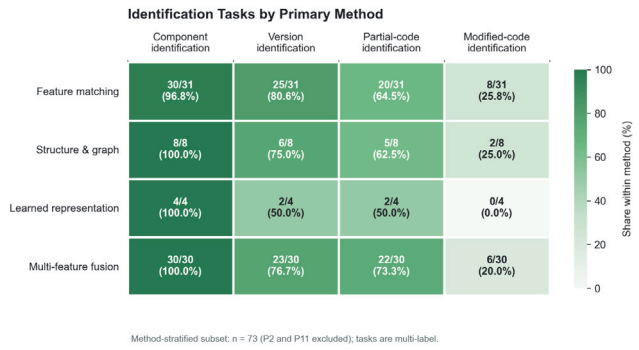


Fig. 6. Cross-distribution of primary methods and research tasks ($n = 73$).

SELF (Liu et al., 2021) retrieves functions using vectors of opcode co-occurrences. The fourth study applies learned analysis to firmware-oriented SBOM recovery (Beninger et al., 2024). Their conclusions depend on the training samples, data split, and target distribution. Allowing the same project or neighboring versions to cross the training–test boundary can also create overlap. All four studies use `Closed-World`, and none treats modified-code identification as a task. Current evidence does not show that these approaches are better suited to out-of-reference targets or modified-code reuse.

Multi-feature fusion. Foiegras (Gousios et al., 2026) combines segment hashes and embeddings; BinaryAI (Jiang et al., 2024) combines function representations, linkage locality, and call relations. LibAM (S. Li et al., 2024), LibvDiff (Dong et al., 2024), and ModX (Yang et al., 2022) fuse several evidence types for regions, version differences, and partial import, respectively. Source-oriented systems in this family cover dependency recovery and adaptive versioning (Tang et al., 2022b; Choi and Woo, 2025; Na et al., 2024). Other systems address binary reuse, complex reuse relations, or cross-build matching (Li et al., 2023; Ng and Prakash, 2013; Yu et al., 2025; Xu et al., 2021; Pizzolotto and Inoue, 2022; Carrillo-Mondejar and Rodriguez, 2025; A. Jia et al., 2025; Ban et al., 2021), while a further group applies identification to security updates, vulnerability analysis, licensing, and firmware studies (Almanee et al., 2021; Y. Hu et al., 2026; Zhao et al., 2024; Feng et al., 2019a; Yuan et al., 2024; X. Guo et al., 2023; Cheng et al., 2023; J. Guo et al., 2023). Fusion can exploit complementary signals, but it cannot recover candidates eliminated early in the pipeline. Weights, thresholds, and stage-wise error propagation also affect the final label. Only six of the 30 studies support unknown rejection; 12 do not and 12 do not report it. Six address modified-code identification.

4.3. Cross-method comparison and evidential limits

Fig. 6 shows task coverage by method family. Every family addresses component, version, and partial-code identification, but modified-code identification is much less frequent. None of the four learned-representation studies treats modified-code identification as a task. The counts indicate coverage, not comparative performance.

Input modes and evaluation settings further constrain comparison. Fig. 7(a) shows that S–B is common in feature matching and multi-feature fusion, while all four learned-representation studies target binaries. S–S can exploit source names and structure and is not directly comparable with B–B under symbol stripping.

Fig. 7(b) and (c) report evaluation environment and unknown handling. These are separate, mutually exclusive coding dimensions. The former describes the relationship between test targets and the reference corpus; the latter records whether the final component-level decision can reject all candidates. An `OpenWorld` data source cannot substitute for an `Unknown` output. Structural and graph matching and learned representations are evaluated mainly in `ClosedWorld`. Among feature-matching and multi-feature-fusion studies, most either do not support unknown handling or do not report it.

Fig. 8 reports whether studies in each family actually evaluate a given variation. Multi-feature-fusion studies cover cross-architecture, cross-optimization, and symbol stripping more often than the other groups. Coverage does not mean that a method successfully handles the variation, and evaluation with out-of-reference targets does not necessarily imply an `Unknown` output.

Figs. 6–8 use each method class as the within-group denominator. P2 (Liu et al., 2026) and P11 (Wang et al., 2025) remain in the 75-study corpus but are excluded from method-class comparisons. Their evaluation evidence is still used in Section 6.

Answer to RQ2. Existing methods produce component, version, and source-fragment results through deterministic features, structural relations, learned representations, or a fusion of several evidence types. Each family has distinct information requirements and error sources. The available evidence does not establish any family as uniformly superior across tasks, input modes, and evaluation settings. Reference coverage, feature observability, evidence aggregation, and unknown rejection jointly delimit practical applicability.

5. Identification risks, limitations, and failure conditions

To examine the gap between controlled experiments and deployment, this section synthesizes five risks from the full texts of the 75 papers: incomplete reference coverage, version and variant ambiguity, partial-code boundaries, code-representation changes, and decision-rule sensitivity. We distinguish direct experimental evidence, author-reported limitations, and analytical judgments made in this review. The analysis considers both the trigger and observed effect of each risk, as well as interactions that may progressively remove useful evidence and produce an incorrect decision. Counts describe research coverage, not risk severity. This section and Section 6 jointly answer RQ3.

5.1. Evidence rules and overall distribution

The five risks were not generated by statistical clustering or formal inference, nor do they reproduce a taxonomy used uniformly in the primary studies. They are an experience-based synthesis developed by the review authors after extensive reading and repeated comparison of recurring failure phenomena. We fixed these dimensions before the final uniform coding and then traced evidence for each dimension through all 75 full texts. The framework organizes the available evidence; it is not claimed to be exhaustive or unique.

We use five mutually exclusive evidence states. A controlled experiment, error analysis, or quantitative result that observes both a condition and its effect is coded as *direct experimental evidence*. An explicit author statement without a corresponding experiment is an *author-reported limitation*. A dependency inferred only from the method, data source, or experimental design is a *review analysis*. If a paper explicitly does not examine the issue, the state is *not evaluated*; if the text is insufficient for a decision, it is *insufficient information*. Fig. 9 selects one primary state per study and risk, using the precedence order just listed. Task labels, method names, and keywords do not substitute for risk evidence.

Fig. 9 is not a severity ranking. The amount of direct evidence depends on the research task, experimental design, and reporting detail. Insufficient information does not mean that a risk is absent, and an author-reported limitation does not mean that its effect has been measured.

The risks frequently co-occur. Among the 57 studies with direct evidence for at least one risk, 13 evaluate one risk, 12 evaluate two, 16 evaluate three, seven evaluate four, and nine evaluate all five. Partial-code reuse, code-representation changes, and a fixed threshold can occur in one pipeline and progressively reduce usable evidence. We therefore discuss both individual risks and their interactions.

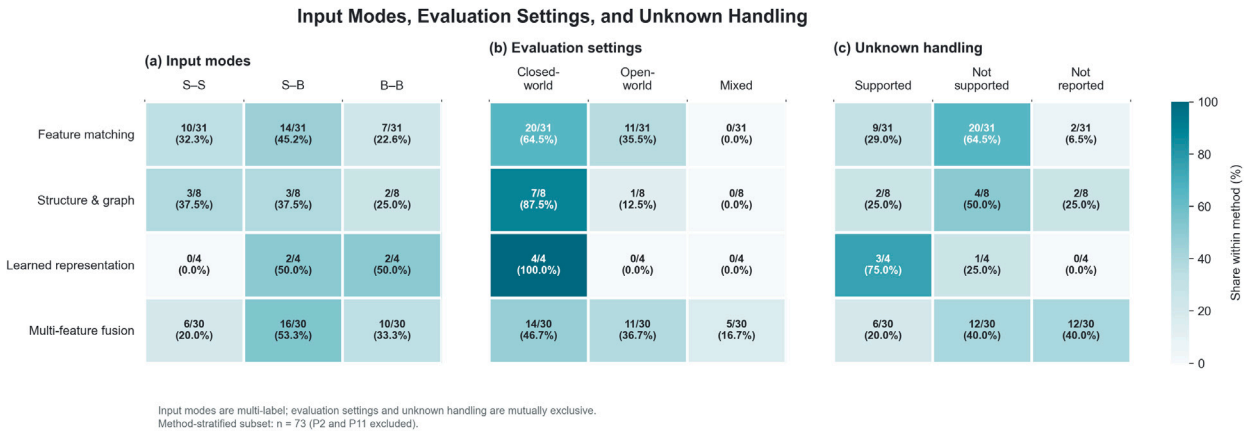


Fig. 7. Input modes, evaluation environments, and unknown handling by primary method ($n = 73$).

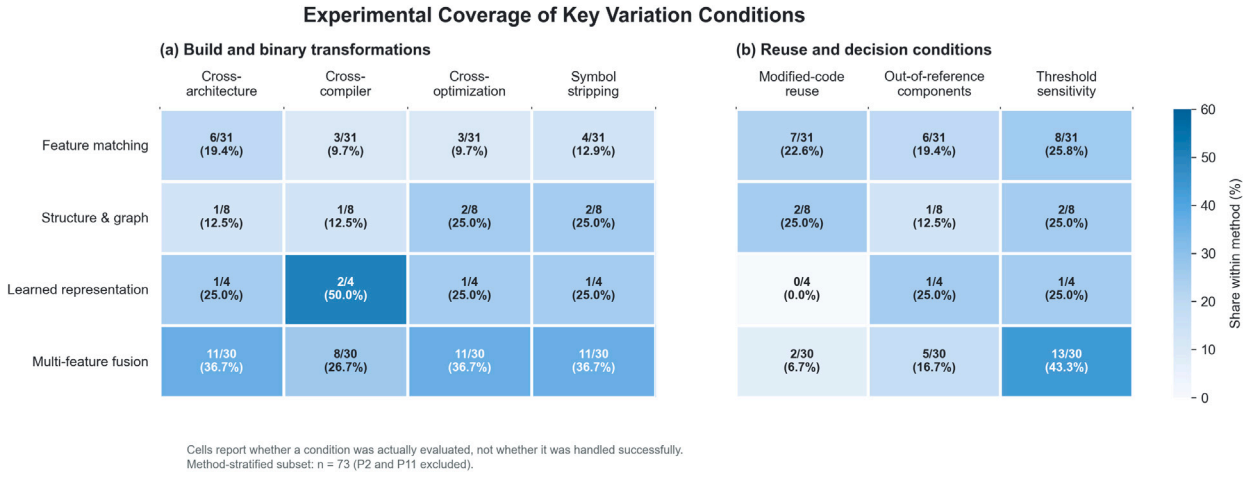


Fig. 8. Experimental coverage of build, reuse, and decision conditions by primary method ($n = 73$).

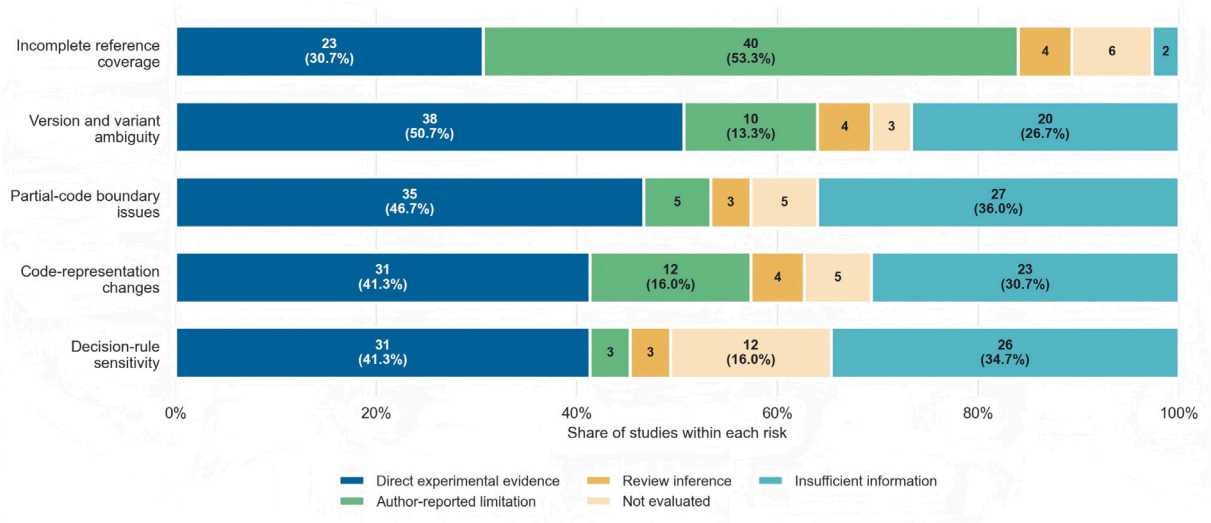


Fig. 9. Study-level primary evidence status for the five identification risks (mutually exclusive coding, $n = 75$).

5.2. Triggers, effects, and available evidence

Incomplete reference coverage. This risk arises when the reference corpus omits the true component, target version, vendor branch, or nested dependency, or when it records their labels and relationships incompletely. An identification system can generally propose only known candidates. If the true source is absent, the system may fail in either direction: it may return no source and create a false negative, or it may force the target onto the closest known component and create a

false positive. Nested dependencies and shared code amplify the second failure. If the component that actually contributed a common function is absent, a higher-level project containing the same function may be reported as its source.

Twenty-three studies directly observe an effect of reference coverage. On real projects with dependencies outside its index, Foiegras (Gousios et al., 2026) reports a drop in library-level accuracy from 85.1% to 70.0%. OSSFP (Wu et al., 2023) attributes 53 false positives to nested-source confusion caused by missing libraries. In P5 (Jiang et al., 2023), 156 of 178 false positives are associated with third-party libraries absent from the dataset. Reference omissions can therefore cause both missed detections and false attribution to an existing candidate. Simply enlarging the corpus does not remove the problem: additional similar versions, shared code, and nested dependencies also increase candidate ambiguity.

Reference completeness must be considered together with unknown handling. Twenty-one studies support a final component-level `Unknown`, no-match, or rejection output; 37 do not, and 17 do not report it. `ClosedWorld` accuracy cannot show whether a system will reject all candidates when the actual source is absent.

Version and variant ambiguity. This risk follows from continuous component evolution. Neighboring versions share most of their code, while the distinguishing evidence may be confined to a few functions, commits, or patches. Version labels are also inconsistent across release tags, package revisions, and vendor editions. Private branches, backported patches, and the coexistence of several versions can make a single "correct version" an unsuitable ground truth. Global similarity may confirm component identity yet rank several neighboring versions together, or build variation may obscure version-specific differences.

Thirty-eight studies provide direct evidence for version or variant discrimination. When P2 (Liu et al., 2026) changes the versions used for testing, LibDX recall decreases from 59% to 30%. P96 (M. Xu et al., 2025) reports that 217 of 376 incorrect-version cases for the baselines involve small feature differences between neighboring versions. LibvDiff (Dong et al., 2024) further notes that a source-level version change that does not reach the final binary leaves no binary evidence to distinguish the release. Foiegras (Gousios et al., 2026) reports lower precision and recall for exact versions than for components, but its comparison does not isolate neighboring releases, branches, or patch ambiguity. We therefore treat it as an indication that version identification is harder, not as direct evidence for this specific risk.

Of the 57 version-identification studies, 42 guarantee that the correct version is present, and 37 use `ClosedWorld`. Such experiments measure ranking or classification among known candidates. They do not answer whether a method can reject an absent version, attribute a vendor branch, or determine the state of a backported patch.

Partial-code boundaries. This risk arises when a target imports only part of a component or when compilation, linking, and project organization split, merge, or hide the original boundaries. Partial-code reuse reduces visible evidence and can cause coverage thresholds designed for complete components to miss a true reuse. Nested dependencies, shared files, and common functions may point to several sources and cause false positives. In binaries, static linking, function inlining, and dead-code elimination can also prevent direct recovery of source files, modules, and function boundaries.

Thirty-five studies directly evaluate this problem. P2 (Liu et al., 2026) finds that 84.1% of its binaries retain less than 20% of their source features and associates low retention with missed detections. In its direct comparison under partial-code reuse, the OSSFP study reports recall of 92.38% for OSSFP and 57.07% for CENTRIS, whose fixed component-ratio threshold misses more cases of partial-code reuse (Wu et al., 2023). The difference shows that aggregation rules materially affect the final result. Of 803 false negatives in P5 (Jiang et al., 2023), 615 come from black-box dependencies whose source code is absent from the target. Together, these results point to both insufficient visible evidence and ambiguous source boundaries.

Lowering a threshold may recover small fragments but also admits more false positives from common functions and shared dependencies. Local provenance evidence must therefore be separated from a complete-component conclusion, and common code, nested dependencies, and multiple-source relations need explicit treatment.

Code-representation changes. These changes include source modification and the effects of compiler, optimization level, architecture, symbol stripping, inlining, linking, and obfuscation. They operate at different layers. Source changes alter the reused content itself. Compilation and architecture alter its implementation representation. Stripping removes names; optimization and inlining restructure functions and control flow; static linking mixes component boundaries. These conditions should not be collapsed into a generic "variant", and robustness to one does not establish robustness to the others.

Thirty-one studies provide direct experiments, but direct evidence is not necessarily comparable evidence. Our full-text recheck found five studies with within-study before–after comparisons at the final identification-task level under consistent methods, data, output granularity, and metrics. Seven provide only controlled function-level or intermediate comparisons; 15 report values under a variation, failure composition, or aggregate results over mixed data; four explain a failure mechanism without comparable numbers. Table 6 reports the five final-task comparisons. Every difference is calculated within its original paper; no metric is pooled across papers.

The paired results show that an effect depends on both the type of change and the feature level. In DeRed, architecture or optimization alone causes a modest decrease, while their combination causes a larger one. BinShape is stable under RR and IR but degrades under DCI and SUB. TPLADD is more stable than OSSFP under the evaluated Type-II modification, and BBDetector's semantic features degrade less than B2SFinder's syntactic features in that experiment. The other 26 papers still provide useful evidence. For example, P2 (Liu et al., 2026) associates 94.17% of false negatives with binaries retaining less than 10% of source features; P5 (Jiang et al., 2023) observes that TLSH distances for modified functions can exceed the cutoff; and LibAM (S. Li et al., 2024) reports region-detection recall of 0.462 in a difficult cross-build setting. These values concern failure attribution, distance thresholds, function or region measures, or unpaired datasets and cannot be ranked together with Table 6. Supplementary Sections S6-3 and S6-4 provide the condition, original result, page evidence, and reason for non-comparability for all 31 studies.

Decision-rule sensitivity. This risk arises after features have been matched. Systems often choose a similarity threshold, minimum match count, coverage ratio, Top-K, clustering parameter, candidate-set size, or multi-feature weight before converting local evidence into component and version labels. A rule optimized for the original validation set may change precision and recall as the reference corpus grows, targets shrink, reuse becomes more partial, or unknown samples are introduced. A forced-matching system always has a Top-1 candidate, even when every score is low. The existence of that candidate is not evidence that it is reliable.

Thirty-one studies directly evaluate decision sensitivity. P2 reports that reducing the database raises OSSPolice precision from 62% to 87% (Liu et al., 2026). P5 reports that lowering the CENTRIS threshold from 0.1 to 0.01 raises recall from 49.44% to 69.35% but lowers precision from 35.71% to 17.65% (Jiang et al., 2023). The LibAM study reports F1 ranges of 0.708–0.996 and 0.322–0.842 on two datasets as its anchor threshold changes (S. Li et al., 2024). Reporting only the best F1 at a selected threshold is therefore insufficient to establish stability.

Table 7 summarizes the triggers and errors for the five risks. Section 7 turns them into evaluation requirements.

Table 6
Within-study paired comparisons of final identification results under code-representation changes.

Study	Variation	Baseline result	Result after variation	Within-study change
P500 (Wang et al., 2024)	GCC build (S1) → MSVC cross-platform build (S3)	Component-level F1 for five B–B methods: 0.117, 0.169, 0.195, 0.271, 0.365	0, 0.010, 0.020, 0.038, 0.168	Decreases of 0.117, 0.159, 0.175, 0.233, and 0.197
DeRed (Chang et al., 2026)	NATIVE (same architecture and optimization) → cross-architecture, cross-optimization, and both combined	F1 = 97.7%	92.9%, 92.0%, 78.1%	Decreases of 4.8, 5.7, and 19.6 percentage points
BinShape (Shirani et al., 2017)	Unobfuscated → register reassignment (RR), instruction reordering (IR), dead-code insertion (DCI), and instruction substitution (SUB)	F1 = 0.89	0.89, 0.89, 0.88, 0.86	Decreases of 0, 0, 0.01, and 0.03
TPLADD (Y. Jia et al., 2025)	Type-II modification ratio $\theta = 0 \rightarrow 0.2$ (variable renaming)	OSSFP F1 = 0.76	OSSFP F1 = 0.32; TPLADD is reported as stable across θ	OSSFP decreases by 0.44; no pointwise TPLADD values, so no difference is calculated
BBDetector (Zhu et al., 2023)	Unobfuscated → obfuscated	BBDetector F1 = 77%; B2SFinder F1 = 67%	71%; 28%	Decreases of 6 and 39 percentage points

Table 7
Mechanisms and error manifestations of the five risks.

Risk	Typical trigger	Main error manifestation
Incomplete reference coverage	Missing components, versions, private branches, or nested dependencies; incomplete labels or relations	False negatives, forced misattribution, incorrect component or version, and omitted vulnerability mapping
Version and variant ambiguity	Small differences between adjacent versions, backported patches, private branches, or coexisting versions	Neighboring-version confusion, unstable labels, and incorrect patch status
Partial-code boundaries	Partial import, trimming, shared code, nested dependencies, static linking, or inlining	False negatives from insufficient coverage, false positives from common code, and incorrect provenance boundaries
Code-representation changes	Source edits, compiler, optimization, architecture, stripping, obfuscation, or linking changes	Feature loss, structural change, reduced similarity, and candidate displacement
Decision-rule sensitivity	Changes in thresholds, Top-K, candidate scale, clustering, weights, or aggregation	Precision–recall tradeoffs, forced labels, and unstable cross-dataset results

5.3. Risk interactions, reporting limitations, and failure chains

The evidence suggests several recurring failure chains. If the true component is absent from the reference corpus and a system still returns the nearest neighbor or Top-1 candidate, an incorrect attribution can be forced. If only a small part of a component remains, source modification or build variation can remove more evidence, after which a fixed coverage threshold may reject the true component. When adjacent versions differ little, build-induced representation changes can obscure the remaining version evidence. If the actual source of shared code is absent from the corpus, another project containing that code may be reported instead. These chains synthesize study-level evidence; they do not imply that every paper experimentally covers every link.

The interactions also show why one aggregate metric may conceal failure conditions. A dataset that guarantees the correct component and version, contains no out-of-reference targets, uses one build configuration, and tests only complete components does not evaluate these chains, even if F1 is high. Conversely, a performance decrease on cross-architecture data cannot be attributed to a generally non-robust algorithm without checking how reference binaries were built, whether targets contain partial-code reuse, whether the version ground truth is unique, and whether thresholds were retuned.

The synthesis is constrained by primary-study reporting. Eighteen studies provide no direct experiment for any of the five risks. Some report only overall accuracy, without error cases, condition-specific results, or parameter curves. Definitions of "partial", "modified", "cross-version", and "unknown" also differ, while the same metric name may refer to functions, files, components, or versions. The evidence counts therefore describe what the literature reports, not how often a risk occurs in deployment. Review analyses in Fig. 9 are kept separate from direct evidence to avoid presenting a plausible concern as an experimentally established fact.

In summary, evidence is uneven across the five risks, and partial-code reuse, code-representation changes, and decision-rule sensitivity can interact in the final result. Section 6 next examines how datasets, experimental settings, metrics, and baselines limit the scope of the available evidence.

6. Experimental evaluation and evidence reliability

An experimental result is meaningful only when interpreted with its task, data source, reference scope, build conditions, output granularity, and metric. We assess evidence reliability along five dimensions: datasets and ground truth, the relationship between reference and test data, variation conditions, metrics and baselines, and public research materials. Comparisons are made only where units and experimental conditions align. Existing results generally support claims within their original settings, but evidence remains incomplete for out-of-reference targets, build variation, data overlap, and full reproduction. This section completes the answer to RQ3.

6.1. Datasets, experimental settings, and representativeness

Data may serve as a reference corpus, training set, validation set, test set, or case study, and one project can serve several roles. The use of a real project is therefore neither sufficient for `OpenWorld` nor proof that reference and test data do not overlap. The evaluation environment must still be determined from the target–reference relationship defined in Section 3.2.

`ClosedWorld` is suitable for measuring retrieval, ranking, and classification among known candidates, but it does not establish behavior when the correct component is absent. `OpenWorld` is closer to deployment, yet nested dependencies, vendor patches, and private releases make its ground truth harder to verify. Out-of-reference targets,

Table 8
Scale of the principal experimental data, stratified by data role.

Unit	Reference corpus	Training/validation	Test	Case study
Project or repository	$n = 9$; 10,241 (1832–133,000)	$n = 3$; 230 (13–1864)	$n = 31$; 100 (9–29,416)	$n = 2$; 3 (1–5)
Component or library	$n = 10$; 648.5 (8–19,057)	$n = 3$; 1192 (24–17,165)	$n = 27$; 896 (4–43,761)	$n = 3$; 33 (4–66)
Component version	$n = 7$; 77,308 (904–726,074)	$n = 1$; 17,918	$n = 10$; 632 (6–229,326)	$n = 1$; 47,015
Function or procedure	$n = 5$; 6,000,000 (7675–56,342,179)	$n = 2$; 11,900,000 (10,000,000–13,800,000)	$n = 13$; 1,130,467 (19,779–2,346,005,582)	$n = 1$; 2761
Binary or executable	$n = 2$; 15,510.5 (5227–25,794)	$n = 5$; 13,800 (524–22,100)	$n = 38$; 888 (5–1,583,732)	–
Firmware image	$n = 1$; 164	$n = 1$; 200	$n = 11$; 10,161 (1–34,136)	$n = 1$; 351
Application or software	–	–	$n = 13$; 64 (5–1,600,000)	–

Cells report the number of studies, median, and minimum–maximum range. The range is omitted when only one study contributes.

partial-code boundaries, and version relations must still be checked explicitly.

Dataset size must be interpreted by both measurement unit and data role. For Table 8, we first take the maximum reported value within the same paper, role, and unit, and then summarize reference, training/validation, test, and case-study data separately. A combined training–test dataset may contribute to both relevant columns, but records coded only as `Mixed` and impossible to separate are omitted from the main distribution. Android native-library files and binaries extracted from firmware are counted as binaries. Function pairs are not merged with individual functions. Complete records, less frequent units, and unclassified roles appear in the supplementary material.

Table 8 shows that the scale of a given unit differs by several orders of magnitude across data roles. A single maximum would conflate reference coverage with the strength of test evidence. Scale is also not representativeness. Millions of samples from one ecosystem or build process may omit private branches, realistic edits, different toolchains, and out-of-reference targets. A smaller, manually verified dataset may be more suitable for a particular risk.

Build and reuse conditions determine how far controlled results can be generalized. Fig. 10 counts studies that explicitly evaluate each condition. Conditions are multi-label, and a paper may appear in several categories. “Evaluated” means that the paper provides a corresponding experiment or error analysis; it does not mean that the result is favorable or that different studies apply changes of equal strength.

Only about one quarter of studies evaluate variation across architectures or optimization levels, or the effect of symbol stripping. Fewer than one fifth evaluate compiler variation, and only five evaluate function inlining. A method that targets binaries cannot therefore be assumed to cover common build changes. Sixteen papers claim modified-code identification as a task, but only eight evaluate modified-code reuse; three additional papers conduct such an experiment without claiming the task, yielding 11 experiments in total. Unknown handling shows the same distinction: 21 systems can return `Unknown` or rejection, but only 13 actually evaluate an unknown-component condition. Capability fields describe what a system can output; variation fields describe what the study tests. They must remain separate.

6.2. Metrics, baselines, and cross-study comparability

Precision, recall, and F1 are the most frequent metrics, appearing in 53, 51, and 32 studies, respectively. Accuracy appears in 15, and 41 report efficiency through runtime, throughput, latency, or a similar measure. These are study counts, not numbers of individual results. A paper often reports the same metric for several datasets, methods, and output levels, so the full metric table contains far more than 75 records.

Metrics with the same name are comparable only when ground truth and prediction use the same output granularity. Accurate function matches do not guarantee correct component aggregation; a correct component name does not guarantee the right version, local boundary, or vulnerability status. Class imbalance can also allow accuracy to conceal missed detections. Each metric should therefore be linked to its output level, dataset, and evaluation environment. Retrieval studies should report candidate measures such as Top-K, while unknown rejection should be evaluated separately from known-component identification.

Efficiency likewise requires context. Offline corpus construction, feature extraction, indexing, a single-target query, firmware unpacking, and batch scanning are different stages. Seconds, minutes, throughput, memory, and index size cannot be combined into one efficiency figure. A fast query may depend on expensive unreported preprocessing, while a longer end-to-end firmware time may cover a more complete workflow. Fair comparison requires, at minimum, aligned hardware, parallelism, reference-corpus size, target unit, and timing boundary.

Sixty-nine studies record at least one experimental baseline. Frequently used baselines include B2SFinder (Feng et al., 2019b) in 14 studies, OSSPolice (Duan et al., 2017) in 12, CENTRIS (Woo et al., 2021) and BinaryAI (Jiang et al., 2024) in 11 each, BAT in eight, and LibDB (Tang et al., 2022a) in five. One included study also directly compares software composition analysis (SCA) tools for C/C++ binaries (Ning et al., 2024). These tools differ in input form, target granularity, reference corpus, and output capability. Their suitability must be judged against the task and available input rather than their popularity.

A fair baseline comparison aligns the task, output granularity, input information, target data, reference corpus, and ground truth, and tunes configurations under a disclosed protocol. If a baseline cannot be run and its published number is quoted instead, this should be explicit. Multi-stage systems also need ablation studies to isolate the contribution of each stage. We therefore do not construct a cross-paper leaderboard of highest accuracy. Where common conditions are absent, we compare only scope and evidence coverage.

6.3. Public materials, reproducibility, and evidence credibility

Reproducibility requires more than source code. It also depends on target and reference data, ground truth, data splits, the build environment, implementation, and key configurations. Learning-based studies additionally need the training procedure, while firmware studies need the corresponding processing workflow. Supplementary Section S5 provides a minimum reporting checklist.

Fig. 11 summarizes materials for which the included studies provide explicit public evidence. Categories are multi-label. Public models or model weights use the 27 studies for which that field is applicable as the denominator; all other categories use the complete set of 75.

Fig. 11 uses a strict criterion of explicit public evidence. A tool or implementation can be source code, an executable, or an online service and does not necessarily mean that all code is open source. Thirty-four studies provide a tool or implementation entry point and 31 provide a dataset. Only 17 clearly release ground truth, and five release a signature corpus or index. Our coding found no study that clearly released a complete set of build scripts or trained models and weights. This does not mean that every remaining study is irreproducible. It shows that the materials needed to reconstruct the reference space and recompute the reported metrics are often incomplete.

Evidence credibility also depends on the origin of ground truth. Component and function mappings generated by a known build process usually have strong internal consistency but remain close to `Closed-World`. Labels generated from package configuration, version strings,

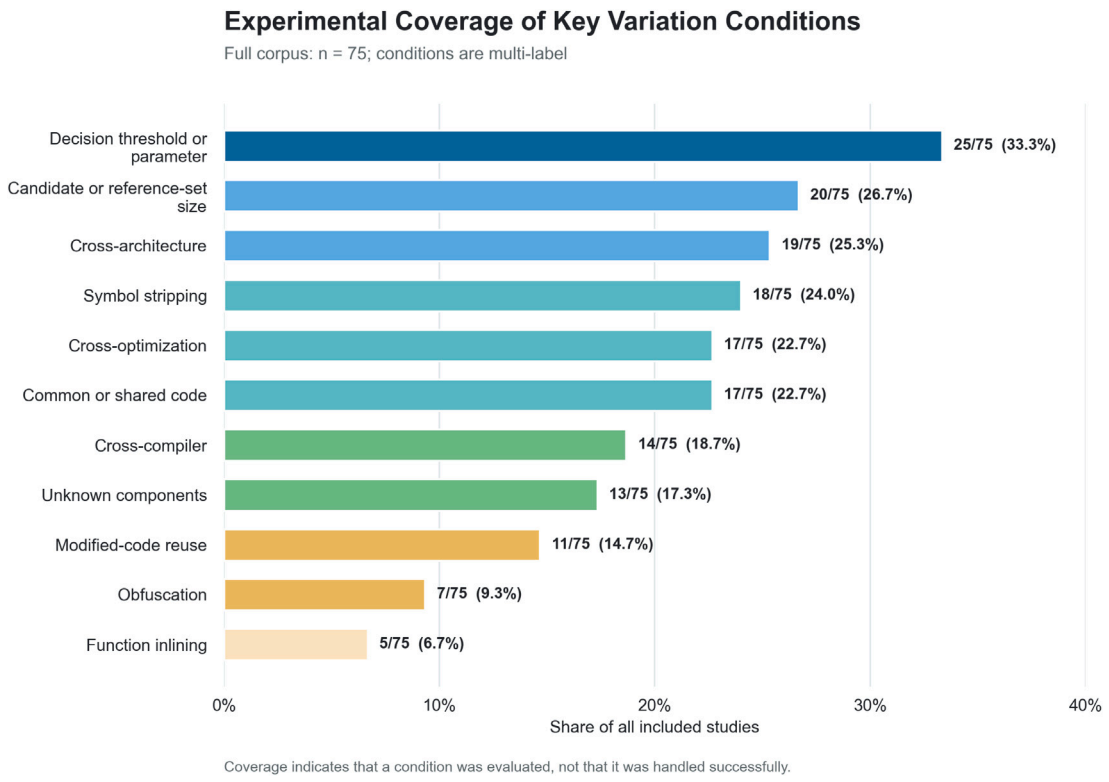


Fig. 10. Experimental coverage of key variation conditions (multi-label coding, $n = 75$).

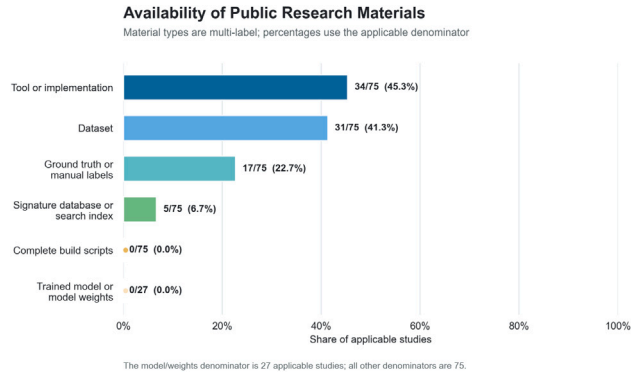


Fig. 11. Public research materials in the included studies (multi-label coding).

or project metadata are inexpensive, yet may not establish that the code reached the final binary. Manually reverse-engineered labels for deployed firmware better represent deployment, but may be incomplete and difficult to audit. A strong evaluation should combine automatically verifiable truth with manual auditing and report annotators, disagreement resolution, unconfirmed samples, and label confidence.

Overlap between training and reference data is another threat. When a target is taken directly from the reference index, the experiment measures in-corpus retrieval. When the same project, neighboring versions, or related builds cross the training–test boundary, the result may also contain leakage. Data should be separated by project, component family, release time, or source repository, and the overlap between the reference corpus and test targets should be reported. This concern applies to non-learning methods as well.

Answer to RQ3. The literature provides varying degrees of study-level evidence for incomplete reference coverage, version and variant ambiguity, partial-code boundaries, code-representation changes, and decision-rule sensitivity. Coverage is nevertheless uneven. Most performance evidence still comes from known candidates and a limited range of build conditions. Unknown components, modified-code reuse, cross-build variation, and complete reproducibility materials receive inadequate evaluation. Existing results generally support effectiveness only under the settings of the original experiments; they should not be generalized directly to deployment where the reference corpus is incomplete or the target has been modified or built under different conditions.

7. Future research directions

The gaps identified in Sections 5 and 6 call for a shift from optimizing a single metric on controlled data to establishing applicability under out-of-reference targets, realistic modifications, and cross-build conditions. This section answers RQ4 through three priorities: identification in open environments, provenance under complex reuse, and a unified evaluation protocol.

7.1. Component, version, and provenance identification in open environments

Open-set decisions should become a standard part of component identification. As shown in Section 6, experimental coverage of unknown components remains limited even among systems that can return `Unknown`, no match, or rejection. Other systems may return the highest-scoring known candidate even when the actual source is absent. A system should distinguish a well-supported known component, a target for which the evidence supports no known candidate, and several candidates that cannot be separated. Rejection rules should be calibrated against reference-corpus size and the amount of target evidence. Function similarity should not be treated directly as component-level confidence.

Reference corpora should also evolve from flat candidate lists into component knowledge bases that represent dependency, inheritance,

fork, release, commit, and patch relationships. Such relations can explain multiple candidates caused by shared code, nested dependencies, and adjacent releases. Version identification should similarly move from a single-label prediction toward evolutionary inference. For a private modification, backported patch, or mixture of versions, a system could report the nearest upstream version and its differences, a version interval, or a patch state instead of forcing one official release label.

Component and version outputs should be accompanied by auditable files, functions, structural regions, or difference fragments. Evidence common to a component should be distinguished from version-specific evidence and from code that may come from a nested dependency. If the true source is missing from the reference corpus, local similarity can support a provenance relation, but it cannot establish that a complete component was imported.

Code-based identification should complement reliable dependency and build records when such records are missing or untrustworthy; it should not replace them when they are available.

7.2. Provenance under partial-code reuse, modification, and cross-build variation

Partial-code identification needs to answer three separate questions: which code is supported by provenance evidence, which project or version the evidence indicates, and whether its extent is sufficient for a complete-component conclusion. Methods should model relations across functions, files, modules, regions, and components. They should combine code frequency, evolutionary relations, and contextual evidence and be evaluated separately on small components, shared code, and nested dependencies. Simply lowering a coverage threshold may recover smaller fragments but will also increase false positives.

Evidence for modified-code identification is particularly weak. Sections 3 and 6 show that experimental coverage remains narrower than task claims. Future datasets should contain real vendor patches, private branches, trimming, refactoring, and functional customization. They should distinguish source modifications from build changes and report results by modification type and magnitude. Simple textual perturbations and compiler optimization are not adequate proxies for maintained downstream branches.

Code representations must tolerate compiler, optimization, architecture, stripping, and inlining variation while preserving release- and patch-specific differences. Future studies should separate projects, versions, time periods, and build configurations across training and test data. A similarity score should also be traceable to supporting functions, call regions, or difference fragments.

Multi-feature fusion may connect interpretable evidence with build-tolerant representations, but each stage needs separate reporting. Candidate recall, feature contribution, aggregation weight, and the final unknown decision should be exposed, and ablation should test how errors propagate through the pipeline.

7.3. A task-specific, open, and reproducible evaluation protocol

The literature differs widely in measurement units, reference scope, build conditions, and output granularity. More isolated performance values will not resolve this comparability problem. A common protocol should use related but separately scored tracks, allowing each method to state its supported scope without reducing several capabilities to one accuracy value. Table 9 proposes seven tracks.

Each track should include `ClosedWorld` and `OpenWorld` subsets. The version track should distinguish whether the correct release is present. The partial- and modified-reuse tracks should control reuse ratio and modification magnitude. The cross-build track should use paired builds from the same source. Baseline comparisons should align tasks, inputs, reference data, ground truth, and output granularity, with fixed tool versions and configurations. Data units and reference, training, test, and deployment roles should be reported separately. Supplementary Section S5 provides candidate baselines, a minimum reporting checklist, and detailed requirements for Table 9.

Future benchmarks should use versions and timestamps to prevent test data from entering training, and should release data, ground truth, build and test materials, and failure cases. Reproducibility would then cover not only a peak metric but also the conditions under which a method fails.

Answer to RQ4. Future research should prioritize open-environment identification under unknown and uncertain versions, interpretable provenance under partial-code reuse, modification, and build variation, and a task-specific reproducible evaluation protocol. Methods and evaluations need to advance together if component results are to support reliable SBOM generation, vulnerability mapping, license analysis, and software supply-chain decisions.

8. Conclusion

This review systematically examined third-party component identification in C/C++ source code, native binaries, and firmware. A traceable search, deduplication, screening, and full-text assessment produced a corpus of 75 papers. We analyzed research tasks, input modes, identification methods, failure risks, and experimental evaluation within one framework.

Existing work concentrates on component, version, and partial-code identification. Evaluation remains sparse for unknown components and modified-code reuse. Of the 75 studies, 47 primarily use `ClosedWorld`, 13 evaluate unknown components, and 11 evaluate modified-code reuse. Incomplete reference coverage, version and variant ambiguity, partial-code boundaries, code-representation changes, and decision-rule sensitivity can also interact, making strong results in controlled experiments difficult to generalize. Heterogeneous data units, output granularities, and evaluation settings, together with incomplete ground truth and reproducibility materials, further restrict cross-study comparison.

Future methods should permit `Unknown` or ambiguous outputs, model relations among components, versions, branches, and patches, and provide auditable local evidence for partial-code and modified-code reuse. Evaluations should separately measure component identification, version identification, partial-code identification, modified-code identification, unknown rejection, cross-build stability, and deployment. They should also report reference scope, ground truth, build conditions, metrics, baselines, and public materials consistently. Progress in both methods and evaluation is needed before identification results can reliably support SBOM generation, vulnerability mapping, license compliance, and software supply-chain governance.

CRediT authorship contribution statement

Wei Guo: Writing – original draft, Methodology, Investigation, Conceptualization. **Xixing Li:** Writing – review & editing, Validation, Supervision. **Qiang Wei:** Visualization, Investigation, Formal analysis.

Funding

This work was supported by the National Natural Science Foundation of China (Grant No. U2436205), the Key Science and Technology Program of Henan Province, China (Grant No. 252102320046), and the Postdoctoral Fellowship Program of CPSF (Grant No. GZB20260996).

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Xixing Li reports financial support was provided by the National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 9
Proposed evaluation tracks for C/C++ third-party component identification.

Track	Key test conditions	Principal outputs and metrics
T1 Component identification	Complete components, non-component negatives, and component-level ground truth; separate S–S, S–B, and B–B	Component-level precision, recall, F1, and evidence localization
T2 Version identification	Adjacent versions, branches, vendor revisions, and backported patches	Exact version, Recall@K, version distance, and calibration error
T3 Partial-code identification	Stratified reuse ratios, shared code, nested dependencies, and local boundary ground truth	Fragment provenance, boundary overlap, and component aggregation
T4 Modified-code identification	Real patches, refactoring, and trimming with modification type and magnitude	Stratified identity, version, and fragment metrics and performance change
T5 Unknown-component rejection	Out-of-reference components, private projects, and unindexed versions	Known-component identification, unknown rejection, false acceptance/rejection, and calibration
T6 Cross-build stability	Paired compiler, optimization, architecture, stripping, inlining, linking, and obfuscation conditions	Condition-specific degradation, cross-condition retrieval, efficiency, and failure type
T7 Deployment	Temporally held-out targets not used to construct the reference corpus or training set	Auditable results, manual verification effort, and end-to-end cost

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.cose.2026.105221.

Data availability

Data will be made available on request.

References

Akabane, S., Okamoto, T., 2020. Identification of library functions statically linked to linux malware without symbols. Procedia Comput. Sci. http://dx.doi.org/10.1016/j.procs.2020.09.053.

Akabane, S., Okamoto, T., 2021. Identification of toolchains used to build IoT malware with statically linked libraries. In: Knowledge-Based and Intelligent Information & Engineering Systems. KSE 2021, http://dx.doi.org/10.1016/j.procs.2021.09.291.

Almanee, S., Ünal, A., Payer, M., Garcia, J., 2021. Too quiet in the library: An empirical study of security updates in android apps' native code. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE, http://dx.doi.org/10.1109/ICSE43902.2021.00122.

Ban, G., Xu, L., Xiao, Y., Li, X., Yuan, Z., Huo, W., 2021. B2SMatcher: fine-grained version identification of open-source software in binary files. Cybersecurity http://dx.doi.org/10.1186/s42400-021-00085-7.

Beninger, M., Charland, P., Ding, S.H.H., Fung, B.C.M., 2024. ERS0: Enhancing military cybersecurity with AI-driven SBOM for firmware vulnerability detection and asset management. In: 2024 16th International Conference on Cyber Conflict: Over the Horizon, Cycon. http://dx.doi.org/10.23919/CyCon62501.2024.10685598.

Brereton, P., Kitchenham, B.A., Budgen, D., et al., 2007. Lessons from applying the systematic literature review process within the software engineering domain. J. Syst. Softw. 80 (4), 571–583.

Brumley, D., Jager, I., Avgerinos, T., Schwartz, E.J., 2011. Bap: a binary analysis platform. In: Computer Aided Verification. CAV 2011, In: Lecture Notes in Computer Science, 6806, Springer, pp. 463–469. http://dx.doi.org/10.1007/978-3-642-22110-1_37.

Carrillo-Mondejar, J., Rodriguez, R.J., 2025. Identifying runtime libraries in statically linked linux binaries. Future Gener. Comput. Syst.- Int. J. Escience http://dx.doi.org/10.1016/j.future.2024.107602.

Chang, S., Yang, S., Deng, X., et al., 2026. DeRed: Enhancing third-party library detection in binaries via deceptive reuse mitigation. Comput. Secur. http://dx.doi.org/10.1016/j.cose.2026.104844.

Cheng, Y., Yang, S., Lang, Z., Shi, Z., Sun, L., 2023. VERI: A large-scale open-source components vulnerability detection in IoT firmware. Comput. Secur. http://dx.doi.org/10.1016/j.cose.2022.103068.

Choi, Y., Woo, S., 2025. TIVER: Identifying adaptive versions of C/C++ third-party open-source components using a code clustering technique. In: 2025 IEEE/ACM 47th International Conference on Software Engineering. ICSE, http://dx.doi.org/10.1109/ICSE55347.2025.00188.

David, Y., Yahav, E., 2014. Tracelet-based code search in executables. ACM SIGPLAN Not. 49 (6), 349–360.

Decan, A., Mens, T., Constantinou, E., 2018a. On the evolution of technical lag in the npm package dependency network. In: IEEE International Conference on Software Maintenance and Evolution. ICSME.

Decan, A., Mens, T., Constantinou, E., 2018b. On the impact of security vulnerabilities in the npm package dependency network. In: International Conference on Mining Software Repositories. MSR.

Decan, A., Mens, T., Grosjean, P., 2019. An empirical comparison of dependency network evolution in seven software packaging ecosystems. Empir. Softw. Eng. 24 (1), 381–416.

Dominguez-Alvarez, D., de la Cruz, A., Gorla, A., Caballero, J., 2023. LibKit: Detecting third-party libraries in iOS apps. In: Proceedings of the 31st ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2023, http://dx.doi.org/10.1145/3611643.3616344.

Dong, C., Li, S., Yang, S., et al., 2024. LibvDiff: Library version difference guided OSS version identification in binaries. In: Proceedings of the IEEE/ACM 46th International Conference On Software Engineering. ICSE 2024, http://dx.doi.org/10.1145/3597503.3623336.

Duan, R., Bijlani, A., Xu, M., Kim, T., Lee, W., 2017. Identifying open-source license violation and 1-day security risk at large scale. In: 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS '17, http://dx.doi.org/10.1145/3133956.3134048.

Feng, M., Mao, W., Yuan, Z., et al., 2019a. Open-source license violations of binary software at large scale. In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering. SANER, http://dx.doi.org/10.1109/SANER.2019.8667977.

Feng, M., Yuan, Z., Li, F., et al., 2019b. B2SFinder: Detecting open-source software reuse in COTS software. In: 34th IEEE/ACM International Conference on Automated Software Engineering. ASE 2019, http://dx.doi.org/10.1109/ASE.2019.00100.

Gousios, G., Hamer, P., Odlund, C., et al., 2026. Foiegras: Source code based software composition analysis for C/C++ applications. In: Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice. http://dx.doi.org/10.1145/3786583.3786878.

Guo, X., Cai, R., Yin, X., Shao, W., Liu, S., 2023. Searching open-source vulnerability function based on software modularization. Appl. Sci.-Basel http://dx.doi.org/10.3390/app13020701.

Guo, J., Wang, Y., Wang, Z., 2023. Accurate detection of third-party library version of IoT firmware based on GNN. Front. Artif. Intell. Appl. http://dx.doi.org/10.3233/FAIA230854.

Hai, X., Zeng, J., Fu, Z., et al., 2025. OSSDetector: Towards a more accurate approach for C/C++ third-party library detection. In: 2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications, Trustcom. http://dx.doi.org/10.1109/Trustcom66490.2025.00285.

Haile, J., Havens, S., 2020. Identifying ubiquitious third-party libraries in compiled executables using annotated and translated disassembled code with supervised machine learning. In: 2020 IEEE Symposium on Security and Privacy Workshops. SPW 2020, http://dx.doi.org/10.1109/SPW50608.2020.00042.

Haq, I.U., Caballero, J., 2021. A survey of binary code similarity. ACM Comput. Surv. 54 (3), 51.

He, J., Chen, X., Yao, S., Liang, X., Du, R., 2025. ConCloneDep: A confidence-aware clone-based dependency identification approach in C/C++ open-source components. In: 2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications, Trustcom. http://dx.doi.org/10.1109/Trustcom66490.2025.00103.

Hu, Y.-T., Dong, J.-J., Sun, M.-Y., Zhang, Y.-H., Wu, Y.-M., Zou, D.-Q., 2026. CentSCA: A software composition analysis method for C/C++ project based on centrality analysis. Jisuanji Xuebao/Chin. J. Comput. http://dx.doi.org/10.11897/SP.J.1016.2026.00679.

Hu, Y., Li, C., Wu, Y., Cai, Y., Zou, D., 2026. VulSCA: A Community-Level SCA Approach for Accurate C/C++ Supply Chain Vulnerability Analysis. NDSS.

Hu, X., Zhang, W., Li, H., et al., 2020. VES: A component version extracting system for large-scale IoT firmwares. In: Wireless Algorithms, Systems, and Applications, Pt II. http://dx.doi.org/10.1007/978-3-030-59019-2_5.

Jia, A., Fan, M., Xu, X., Jin, W.-X., Wang, H.-J., Liu, T., 2025. Binary2Source function similarity detection method under function inlining. Ruan Jian Xue Bao/J. Softw. http://dx.doi.org/10.13328/j.cnki.jos.007335.

Jia, Y., Wang, J., Wu, P., 2025. TPLADD: Highly robust and precise C/C++ third-party libraries detection method. J. Front. Comput. Sci. Technol. http://dx.doi.org/10.3778/j.issn.1673-9418.2409052.

Jia, Q., Zhang, M., Guo, X., et al., 2024. Research on open-source component homology analysis model based on graph neural network and attention mechanism. In: 2024 4th International Conference on Consumer Electronics and Computer Engineering. ICCECE, http://dx.doi.org/10.1109/ICCECE61317.2024.10504200.

Jiang, L., An, J., Huang, H., et al., 2024. BinaryAI: Binary software composition analysis via intelligent binary source code matching. In: Proceedings of the IEEE/ACM 46th International Conference On Software Engineering. ICSE 2024, http://dx.doi.org/10.1145/3597503.3639100.

Jiang, L., Yuan, H., Tang, Q., Nie, S., Wu, S., Zhang, Y., 2023. Third-party library dependency for large-scale SCA in the C/C++ ecosystem: How far are we?. In: Proceedings of the 32nd ACM SIGSOFT International Symposium On Software Testing and Analysis. ISSTA 2023, http://dx.doi.org/10.1145/3597926.3598143.

Kalysch, A., Milisterfer, O., Protsenko, M., Müller, T., 2018. Tackling android's native library malware with robust, efficient and accurate similarity measures. In: Proceedings of the 13th International Conference on Availability, Reliability and Security. ARES 2018, Association for Computing Machinery, pp. 58:1–58:10. http://dx.doi.org/10.1145/3230833.3232828.

Kikas, R., Gousios, G., Dumas, M., Pfahl, D., 2017. Structure and evolution of package dependency networks. In: IEEE/ACM International Conference on Mining Software Repositories. MSR.

Kim, D., Cho, S.-j., Park, M., Han, S., 2017. Open source software detection using function parameter based software birthmark. J. Internet Technol. http://dx.doi.org/10.6138/JIT.2017.18.4.20160130.

Kim, J.-W., Choi, M.-J., 2025. A cross-language and cross-binary type approach to binary-source software composition analysis using BM25. Int. J. Inf. Secur. http://dx.doi.org/10.1007/s10207-025-01148-3.

Kim, S., Woo, S., Lee, H., Oh, H., 2017. VUDDY: A scalable approach for vulnerable code clone discovery. In: IEEE Symposium on Security and Privacy.

Kitchenham, B., Brereton, O.P., Budgen, D., et al., 2009. Systematic literature reviews in software engineering: A systematic literature review. Inf. Softw. Technol. 51 (1), 7–15.

Kula, R.G., German, D.M., Ouni, A., et al., 2018. Do developers update their library dependencies?. Empir. Softw. Eng. 23 (1), 384–417.

Li, S., Dong, C., Wang, Y., et al., 2023. LibDI: A direction identification framework for detecting complex reuse relationships in binaries. In: MILCOM 2023 - 2023 IEEE Military Communications Conference. http://dx.doi.org/10.1109/MILCOM58377.2023.10356328.

Li, S., Wang, Y., Dong, C., et al., 2024. LibAM: An area matching framework for detecting third-party libraries in binaries. ACM Trans. Softw. Eng. Methodol. http://dx.doi.org/10.1145/3625294.

Li, R., Zhao, E., Chen, X., Hao, K., Hou, A., Gong, J., 2024. Pre-built dependency graph-based TPLs analysis. In: 2024 4th International Symposium on Computer Technology and Information Science. ISCTIS, http://dx.doi.org/10.1109/ISCTIS63324.2024.10698860.

Liu, X., Cao, S., Cao, Z., Gao, Q., Wan, L., Wang, F., 2021. SELF: A method of searching for library functions in stripped binary code. Comput. Secur. http://dx.doi.org/10.1016/j.cose.2021.102473.

Liu, C., Xu, Z., Li, K., et al., 2026. Understanding the limitations of C/C++ binary third-party library detection tool: An empirical study at scale. In: Proc. ACM Softw. Eng. http://dx.doi.org/10.1145/3808096.

Massarelli, L., Di Luna, G.A., Petroni, F., et al., 2019. SAFE: Self-attentive function embeddings for binary similarity. In: Detection of Intrusions and Malware, and Vulnerability Assessment. DIMVA.

Min, H., Zhang, L., 2019. Survey on software clone detection research. In: 3rd International Conference on Management Engineering, Software Engineering and Service Sciences.

Mirhosseini, S., Parnin, C., 2017. Can automated pull requests encourage software developers to upgrade out-of-date dependencies?. In: IEEE/ACM International Conference on Automated Software Engineering. ASE.

Na, Y., Woo, S., Lee, J., Lee, H., 2024. Cneps: A precise approach for examining dependencies among third-party C/C++ open-source components. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE 2024, http://dx.doi.org/10.1145/3597503.3639209.

Ng, B.H., Prakash, A., 2013. Expose: Discovering potential binary code re-use. In: 2013 IEEE 37th Annual Computer Software and Applications Conference. COMPSAC, http://dx.doi.org/10.1109/COMPSAC.2013.83.

Ning, Y., Zhang, Y., Ma, C., Guo, Z., Yu, L., 2024. Empirical study of software composition analysis tools for C/C++ binary programs. IEEE ACCESS http://dx.doi.org/10.1109/ACCESS.2023.3341224.

Nouh, L., Rahimian, A., Mouheb, D., Debbabi, M., Hanna, A., 2017. BinSign: Fingerprinting binary functions to support automated analysis of code executables. In: ICT Systems Security and Privacy Protection, SEC 2017. http://dx.doi.org/10.1007/978-3-319-58469-0_23.

Page, M.J., et al., 2021. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. Int. J. Surg. 88, 105906.

Pashchenko, I., Plate, H., Ponta, S.E., et al., 2018. Vulnerable open source dependencies: Counting those that matter. In: ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. ESEM.

Petersen, K., Feldt, R., Mujtaba, S., Mattsson, M., 2008. Systematic mapping studies in software engineering. In: 12th International Conference on Evaluation and Assessment in Software Engineering. EASE.

Pewny, J., Garmany, B., Gawlik, R., et al., 2015. Cross-architecture bug search in binary executables. In: IEEE Symposium on Security and Privacy.

Pizzolotto, D., Inoue, K., 2022. BinCC: Scalable function similarity detection in multiple cross-architectural binaries. IEEE ACCESS http://dx.doi.org/10.1109/ACCESS.2022.3225100.

Ponta, S.E., Plate, H., Sabetta, A., 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In: IEEE International Conference on Software Maintenance and Evolution. ICSME.

Qiu, J., Su, X., Ma, P., 2015. Library functions identification in binary code by using graph isomorphism testings. In: 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering. SANER, http://dx.doi.org/10.1109/SANER.2015.7081836.

Qiu, J., Su, X., Ma, P., 2016. Using reduced execution flow graph to identify library functions in binary code. IEEE Trans. Softw. Eng. http://dx.doi.org/10.1109/TSE.2015.2470241.

Reichert, B.M., Obelheiro, R.R., 2024. Software supply chain security: A systematic literature review. Int. J. Comput. Appl. 46, 853–867.

Reid, D., Mockus, A., 2023. Applying the universal version history concept to help de-risk copy-based code reuse. In: 2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation. SCAM, http://dx.doi.org/10.1109/SCAM59687.2023.00012.

Rethlefsen, M.L., Kirtley, S., Waffenschmidt, S., et al., 2021. PRISMA-S: An extension to the PRISMA statement for reporting literature searches in systematic reviews. Syst. Rev. 10 (39).

Rhee, J.J., Zuo, F., Tompkins, C., Oh, J., Choe, Y.R., 2024. GrizzlyBay: Retroactive SBOM with automated identification for legacy binaries. In: MILCOM 2024 - 2024 IEEE Military Communications Conference. MILCOM, http://dx.doi.org/10.1109/MILCOM61039.2024.10773925.

Shirani, P., Wang, L., Debbabi, M., 2017. BinShape: Scalable and robust binary library function identification using function shape. In: Detection of Intrusions and Malware, and Vulnerability Assessment. DIMVA 2017, http://dx.doi.org/10.1007/978-3-319-60876-1_14.

Stojanovic, S., Radivojevic, Z., Cvetanovic, M., 2015. Approach for estimating similarity between procedures in differently compiled binaries. Inf. Softw. Technol. http://dx.doi.org/10.1016/j.infsof.2014.06.012.

Sun, W., Shan, Z., Liu, F., Li, X., Qiao, M., Zhang, C., 2020. Software supply chain analysis based on function similarity. J. Phys.: Conf. Ser. http://dx.doi.org/10.1088/1742-6596/1601/5/052020.

Tang, W., Chen, D., Luo, P., 2018. BCFinder: A lightweight and platform-independent tool to find third-party components in binaries. In: 2018 25th Asia-Pacific Software Engineering Conference. APSEC 2018, http://dx.doi.org/10.1109/APSEC.2018.00043.

Tang, W., Luo, P., Fu, J., Zhang, D., 2020. LibDX: A cross-platform and accurate system to detect third-party libraries in binary code. In: Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution, and Reengineering. SANER '20, http://dx.doi.org/10.1109/saner48275.2020.9054845.

Tang, W., Wang, Y., Zhang, H., Han, S., Luo, P., Zhang, D., 2022a. LibDB: An effective and efficient framework for detecting third-party libraries in binaries. In: 2022 Mining Software Repositories Conference. MSR 2022, http://dx.doi.org/10.1145/3524842.3528442.

Tang, W., Xu, Z., Liu, C., et al., 2022b. Towards understanding third-party library dependency in C/C++ ecosystem. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. ASE 2022, http://dx.doi.org/10.1145/3551349.3560432.

Wang, H., Liu, Z., Dai, Y., et al., 2025. Preserving privacy in software composition analysis: A study of technical solutions and enhancements. In: 2025 IEEE/ACM 47th International Conference on Software Engineering. ICSE, http://dx.doi.org/10.1109/ICSE55347.2025.00055.

Wang, H., Liu, Z., Wang, S., Wang, Y., Tang, Q., Nie, S., Wu, S., 2024. Are we there yet? filling the gap between binary similarity analysis and binary software composition analysis. In: Proceedings of the 9th IEEE European Symposium on Security and Privacy. EuroS&P 2024, IEEE, pp. 506–523. http://dx.doi.org/10.1109/EuroSP60621.2024.00034.

Wittern, E., Suter, P., Rajagopalan, S., 2016. A look at the dynamics of the JavaScript package ecosystem. In: IEEE/ACM International Conference on Mining Software Repositories. MSR.

Woo, S., Choi, E., Lee, H., Oh, H., 2023. V1SCAN: Discovering 1-day vulnerabilities in reused C/C++ open-source software components using code classification techniques. In: 32nd USENIX Security Symposium. USENIX Security 2023.

Woo, S., Hong, H., Choi, E., Lee, H., 2022. MOVERY: A precise approach for modified vulnerable code clone discovery from modified open-source software components. In: USENIX Security Symposium.

Woo, S., Park, S., Kim, S., Lee, H., Oh, H., 2021. CENTRIS: A precise and scalable approach for identifying modified open-source software reuse. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE, http://dx.doi.org/10.1109/ICSE43902.2021.00083.

Wu, J., Xu, Z., Tang, W., et al., 2023. OSSFP: Precise and scalable C/C++ third-party library detection using fingerprinting functions. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, http://dx.doi.org/10.1109/ICSE48619.2023.00034.

Xu, S., Dong, J., Cai, W., Li, J., Shaghaghi, A., Sun, N., Ma, S., 2025. Enhancing security in third-party library reuse - comprehensive detection of 1-day vulnerability through code patch analysis. In: Proceedings of the 32nd Annual Network and Distributed System Security Symposium. NDSS 2025, Internet Society, http://dx.doi.org/10.14722/ndss.2025.240576.

Xu, X., Liu, C., Feng, Q., et al., 2017. Neural network-based graph embedding for cross-platform binary code similarity detection. In: ACM SIGSAC Conference on Computer and Communications Security. CCS.

Xu, M., Wang, Y., Tang, W., et al., 2025. Demystifying cross-language C/C++ binaries: A robust software component analysis approach. In: 2025 40th IEEE/ACM International Conference on Automated Software Engineering. ASE, http://dx.doi.org/10.1109/ASE63991.2025.00148.

Xu, X., Zheng, Q., Yan, Z., Fan, M., Jia, A., Liu, T., 2021. Interpretation-enabled software reuse detection based on a multi-level birthmark model. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE, http://dx.doi.org/10.1109/ICSE43902.2021.00084.

Yang, C., Xu, Z., Chen, H., Liu, Y., Gong, X., Liu, B., 2022. Modx: binary level partially imported third-party library detection via program modularization and semantic matching. In: Proceedings of the 44th International Conference on Software Engineering. ICSE 2022, Association for Computing Machinery, pp. 1393–1405. http://dx.doi.org/10.1145/3510003.3510627.

Yu, W., Li, S., Huang, M., Li, M., Xi, R., Zhu, H., 2025. LibRI: A module analysis framework for identifying complex reuse relationship in binaries. In: 2025 28th International Conference on Computer Supported Cooperative Work in Design. CSCWD, http://dx.doi.org/10.1109/CSCWD64889.2025.11033368.

Yu, B., Zhang, Y., Zhang, Y., Yang, Q., 2023. Firm VEA: Vulnerability discovery optimisation for IoT firmware via version evolution analysis. In: GLOBECOM 2023 - 2023 IEEE Global Communications Conference. http://dx.doi.org/10.1109/GLOBECOM54140.2023.10437669.

Yuan, S., Pu, W., Liu, C., et al., 2024. KPCA for open source security: Analyzing and assessing component vulnerabilities. In: 2024 5th International Conference on Information Science, Parallel and Distributed Systems. ISPDS, http://dx.doi.org/10.1109/ISPDS62779.2024.10667551.

Zerouali, A., Constantinou, E., Mens, T., et al., 2018. An empirical analysis of technical lag in npm package dependencies. In: International Conference on Software Reuse. ICSR.

Zhang, W., Chen, Y., Li, H., Li, Z., Sun, L., 2018. PANDORA: A scalable and efficient scheme to extract version of binaries in IoT firmwares. IEEE Int. Conf. Commun. http://dx.doi.org/10.1109/ICC.2018.8423015.

Zhang, S., Liu, C., Chen, S., Zhang, L., Liu, Y., 2026. Scratching the iceberg: Unveiling the outdated third-party native libraries in android apps. In: Proceedings - 2026 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER 2026, http://dx.doi.org/10.1109/SANER67736.2026.00054.

Zhang, L., Liu, C., Wu, J., et al., 2025. Drop the golden apples: Identifying third-party reuse by DB-less software composition analysis. In: Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering. FSE COMPANION 2025, http://dx.doi.org/10.1145/3696630.3728525.

Zhang, Y., Ma, C., Ning, Y., Wu, Q., Guo, Z., 2024a. Research and implementation of open source component library detection for binary programs. IEEE Access http://dx.doi.org/10.1109/ACCESS.2024.3442191.

Zhang, W.-D., Yin, L.-B., Li, H., Wen, H., Sun, L.-M., 2018. Open source component version recognition without version strings. Ruan Jian Xue Bao/J. Softw..

Zhang, Y., Yu, B., 2023. FirmCVI: Taint analysis-based component version identification method for large-scale IoT firmware. In: 2023 IEEE 29th International Conference on Parallel and Distributed Systems. ICPADS, http://dx.doi.org/10.1109/ICPADS60453.2023.00137.

Zhang, Y., Yu, B., Zhou, L., Yang, Q., 2024b. Comprehensive IoT firmware characterization based on a large-scale firmware dataset. In: ACM International Conference Proceeding Series. http://dx.doi.org/10.1145/3689236.3689262.

Zhao, B., Ji, S., Xu, J., et al., 2024. One bad apple spoils the barrel: Understanding the security risks introduced by third-party components in IoT firmware. IEEE Trans. Dependable Secure Comput. http://dx.doi.org/10.1109/TDSC.2023.3279846.

Zhu, X., Wang, J., Fang, Z., Yin, X., Liu, S., 2023. BBDetector: A precise and scalable third-party library detection in binary executables with fine-grained function-level features. Appl. Sci.-Basel http://dx.doi.org/10.3390/app13010413.

Zimmermann, M., Staicu, C.-A., Tenny, C., Pradel, M., 2019. Small world with high risks: a study of security threats in the npm ecosystem. In: 28th USENIX Security Symposium (USENIX Security 19). USENIX Security 2019, USENIX Association, ISBN: 978-1-939133-06-9, pp. 995–1010.

Zou, Y., Zhang, Y., Zhao, G., Wu, Y., Shen, S., Fu, C., 2025. BinCoFer: Three-stage purification for effective C/C++ binary third-party library detection. JOURNAL OF SYSTEMS AND SOFTWARE http://dx.doi.org/10.1016/j.jss.2025.112480.